

Cyclic Implicit Complexity

GIANLUCA CURZI, Department of Philosophy, Linguistics and Theory of Science, University of Gothenburg, Gothenburg, Sweden

ANUPAM DAS, School of Computer Science, University of Birmingham, Birmingham, United Kingdom

Circular (or *cyclic*) proofs have received increasing attention in recent years and have been proposed as an alternative setting for studying (co)inductive reasoning. In particular, now several type systems based on circular reasoning have been proposed. However, little is known about the complexity theoretic aspects of circular proofs, which exhibit sophisticated loop structures atypical of more common ‘recursion schemes’.

This article attempts to bridge the gap between circular proofs and *implicit computational complexity* (ICC). Namely, we introduce a circular proof system based on Bellantoni and Cook’s famous safe-normal function algebra, and we identify proof theoretical constraints, inspired by ICC, to characterise the polynomial-time and elementary computable functions. Along the way, we introduce new recursion theoretic implicit characterisations of these classes that may be of interest in their own right.

CCS Concepts: • **Theory of computation** → **Complexity theory and logic**; **Proof theory**;

Additional Key Words and Phrases: Cyclic proofs, implicit complexity, function algebras, safe recursion, higher-order complexity

ACM Reference format:

Gianluca Curzi and Anupam Das. 2026. Cyclic Implicit Complexity. *ACM Trans. Comput. Logic* 27, 2, Article 11 (April 2026), 44 pages.

<https://doi.org/10.1145/3793666>

1 Introduction

Formal proofs are traditionally seen as finite objects modelling logical or mathematical reasoning. *Non-wellfounded* proofs are a generalisation of this notion to an infinitary (but finitely branching) setting, in which consistency is maintained by a standard global condition: the ‘progressing’ criterion. Special attention is devoted to *regular* (or *circular* or *cyclic*) proofs, i.e., those non-wellfounded proofs having only finitely many distinct sub-proofs, and which may thus be represented by finite

This work was supported by a UKRI Future Leaders Fellowship, “Structure vs Invariants in Proofs” (project reference MR/S035540/1), by the Wallenberg Academy Fellowship Prolongation project “Taming Jörmungandr: The Logical Foundations of Circularity” (project reference 251080003), and by the VR starting grant “Proofs with Cycles in Computation” (project reference 251088801).

Author’s Contact Information: Gianluca Curzi (corresponding author), Department of Philosophy, Linguistics and Theory of Science, University of Gothenburg, Gothenburg, Sweden; e-mail: gianluca.curzi@gu.se; Anupam Das, School of Computer Science, University of Birmingham, Birmingham, United Kingdom; e-mail: A.Das@bham.ac.uk.



This work is licensed under Creative Commons Attribution International 4.0.

© 2026 Copyright held by the owner/author(s).

ACM 1557-945X/2026/4-ART11

<https://doi.org/10.1145/3793666>

(possibly cyclic) directed graphs. For such proofs the progressing criterion may be effectively decided by reduction to the universality problem for Büchi automata.

Non-wellfounded proofs have been employed to reason about the modal μ -calculus and fixed-point logics [19, 30], first-order inductive definitions [10], Kleene algebra [16, 17], linear logic [3, 20], arithmetic [8, 14, 34], system T [13, 15, 25] and continuous cut-elimination [2, 21, 29]. In particular, Kuperberg et al. [25] and Das [13, 15] investigated the computational expressivity of circular proofs, with respect to the proofs-as-programs paradigm, in the setting of higher-order primitive recursion.

However, little is known about the complexity-theoretic aspects of circular proofs. Usual termination arguments for circularly typed programs are nonconstructive, proceeding by contradiction and using a non-recursive ‘totality’ oracle (cf. [13, 15, 25]). As a result, these arguments are not appropriate for delivering feasible complexity bounds (cf. [14]).

The present article aims to bridge this gap by proposing a circular foundation for Implicit Computational Complexity (ICC), a branch of computational complexity studying machine-free characterisations of complexity classes. Our starting point is Bellantoni and Cook’s famous function algebra B characterising the polynomial time computable functions (**FPTIME**) using *safe recursion* [6]. The prevailing idea behind safe recursion (and its predecessor, ‘ramified’ recursion [26]) is to organise data into strata in a way that prevents recursive calls being substituted into recursive parameters of previously defined functions. This approach has been successfully employed to give resource-bound-free characterisations of polynomial-time [6], levels of the polynomial-time hierarchy [5], and levels of the Grzegorzczuk hierarchy [35], and has been extended to higher-order settings too [22, 27].

Circular Systems for Implicit Complexity. Construing B as a type system, we consider non-wellfounded proofs, or *coderivations*, generated by its recursion-free subsystem B^- . The circular proof system CNB is then obtained by considering the regular and progressing coderivations of B^- which satisfy a further criterion, *safety*, motivated by the eponymous notion from Bellantoni and Cook’s work (cf. also ‘ramification’ in Leivant’s work [26]). On the one hand, regularity and progressiveness ensure that coderivations of CNB define total computable functions; on the other hand, the latter criterion ensures that the corresponding equational programs are ‘safe’: the recursive call of a function is never substituted into the recursive parameter of a step function.

Despite CNB having only ground types, it is able to define equational programs that nest recursive calls, a property that typically arises only in higher-order recursion (cf., e.g., [22, 27]). In fact, we show that this system defines precisely the elementary computable functions (**FELEMENTARY**). Let us point out that the capacity of circular proofs to simulate some higher-order behaviour reflects an emerging pattern in the literature. For instance in [13, 15] it is shown that the number-theoretic functions definable by type level n proofs of a circular version of system T are exactly those definable by type level $n + 1$ proofs of T.

In the setting of ICC, Hofmann [22] and Leivant [27] already observed that higher-order safe recursion mechanisms can be used to characterise **FELEMENTARY**. In particular, Hofmann [22] presents the type system SLR (Safe Linear Recursion) as a higher-order version of B imposing a ‘linearity’ restriction on the higher-order safe recursion operator. He shows that this system defines just the polynomial-time computable functions on natural numbers (**FPTIME**).

Inspired by Hofmann [22], we too introduce a linearity requirement for CNB that is able to control the interplay between cycles and the cut rule, called the *left-leaning criterion*. The resulting circular proof system is called CB, which we show defines precisely **FPTIME**.

Function Algebras for Safe Nested Recursion and Safe Recursion along Well-Founded Relations. As well as introducing the circular systems CB and CNB just mentioned, we also develop novel

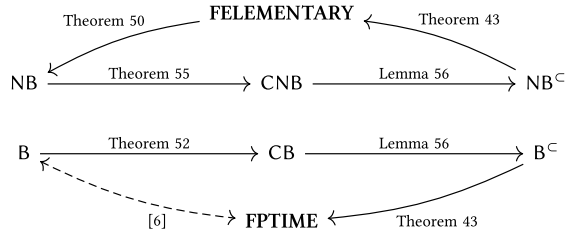


Fig. 1. Summary of the main results of the article, where $\rightarrow$ indicates an inclusion ($\subseteq$) of function classes.

function algebras for **FPTIME** and **FELEMENTARY** that allow us to prove the aforementioned complexity characterisations via a ‘sandwich’ technique, cf. Figure 1. This constitutes a novel (and more direct) approach to reducing circularity to recursion, that crucially takes advantage of safety.

We give a relativised formulation of **B**, i.e., with *oracles*, that allows us to define a form of safe *nested* recursion. The resulting function algebra is called **NB** and is comparable to the type 2 fragment of Leivant’s extension of ramified recursion to finite types [27]. The algebras **B** and **NB** will serve as lower bounds for **CB** and **CNB**, respectively.

The relativised formulation of function algebras also admits a robust notion of (safe) recursion along a well-founded relation. We identify a particular well-founded preorder $\subset$ (‘permutation of prefixes’) whose corresponding safe recursor induces algebras **B^C** and **NB^C** that will serve as upper bounds for **CB** and **CNB**, respectively.

Outline. This article is structured as follows. Section 2 presents **B** as a proof system. In Section 3, we define non-wellfounded proofs and their semantics and present the circular proof systems **CNB** and **CB**. In Section 4, we present the function algebras **NB**, **B^C** and **NB^C**. Section 5 shows that **B^C** captures **FPTIME** (Corollary 44) and that both **NB** and **NB^C** capture **FELEMENTARY** (Corollary 51). These results require a delicate Bounding Lemma (Lemma 41) and an encoding of the elementary functions into **NB** (Theorem 50). In Section 6, we show that any function definable in **B** is also definable in **CB** (Theorem 52), and that any function definable in **NB** is also definable in **CNB** (Theorem 55). Finally, in Section 6.2, we present a translation of **CNB** into **NB^C** that maps **CB** coderivations into **B^C** functions (Lemma 56), by reducing circularity to a form of simultaneous recursion on $\subset$.

The main results of this article are summarised in Figure 1.

Comparison with Conference Version. This article is a full version of the conference paper [11], which appeared in the proceedings of *Logic in Computer Science 2022*. The current version expands upon [11] by including full proofs, as well as many further examples and remarks to aid the narrative.

2 Preliminaries

Bellantoni and Cook introduced in [6] an algebra of functions based on a simple two-sorted structure. This idea was itself inspired by Leivant’s characterisations, one of the founding works in ICC [26]. The resulting ‘tiering’ of the underlying sorts has been a recurring theme in the ICC literature since, and so it is this structure that shall form the basis of the systems we consider in this work.

We consider functions on the natural numbers with inputs distinguished into two sorts: ‘safe’ and ‘normal’. We shall write functions explicitly indicating inputs, namely writing $f(x_1, \dots, x_m; y_1, \dots, y_n)$ when f takes m normal inputs $\vec{x}$ and n safe inputs $\vec{y}$. Both sorts vary over the natural

numbers, but their roles will be distinguished by the closure operations of the algebras and rules of the systems we consider. We will sometimes call these functions *safe-normal*.

Throughout this work, we write $|x|$ for the length of x (in binary notation), and if $\vec{x} = x_1, \dots, x_m$ we write $|\vec{x}|$ for the list $|x_1|, \dots, |x_m|$.

2.1 Bellantoni–Cook Characterisation of FPTIME

We first recall Bellantoni–Cook in its original guise.

Definition 1 (Bellantoni–Cook algebra). B is defined as the smallest class of (two-sorted) functions containing:

- $0(\cdot) := 0 \in \mathbb{N}$.
- $\pi_{j;n}^{m;n}(x_1, \dots, x_m; y_1, \dots, y_n) := x_j$, for $1 \leq j \leq m$.
- $\pi_{j;n}^{m;n}(x_1, \dots, x_m; y_1, \dots, y_n) := y_j$, for $1 \leq j \leq n$.
- $s_i(\cdot; x) := 2x + i$, for $i \in \{0, 1\}$.
- $p(\cdot; x) := \lfloor x/2 \rfloor$.
- $\text{cond}(\cdot; w, x, y, z) := \begin{cases} x & w = 0 \\ y & w = 0 \pmod{2}, w \neq 0 \\ z & w = 1 \pmod{2} \end{cases}$

and closed under the following:

- (Safe composition)
 - If $f(\vec{x}; x; \vec{y}), g(\vec{x}; \cdot) \in B$ then $f(\vec{x}, g(\vec{x}; \cdot); \vec{y}) \in B$.
 - If $f(\vec{x}; \vec{y}, y), g(\vec{x}; \vec{y}) \in B$ then $f(\vec{x}; \vec{y}, g(\vec{x}; \vec{y})) \in B$.
- (Safe recursion on notation) If $g(\vec{x}; \vec{y}), h_i(x, \vec{x}; \vec{y}, y) \in B$ for $i = 0, 1$ then so is $f(x, \vec{x}; \vec{y})$ given by:

$$\begin{aligned} f(0, \vec{x}; \vec{y}) &:= g(\vec{x}; \vec{y}) \\ f(s_0x, \vec{x}; \vec{y}) &:= h_0(x, \vec{x}; \vec{y}, f(x, \vec{x}; \vec{y})) \text{ if } x \neq 0 \\ f(s_1x, \vec{x}; \vec{y}) &:= h_1(x, \vec{x}; \vec{y}, f(x, \vec{x}; \vec{y})). \end{aligned}$$

Intuitively, in a function $f(\vec{x}; \vec{y}) \in B$ only the normal arguments $\vec{x}$ can be used as recursive parameters. The idea behind safe recursion is that recursive calls can only appear in safe position, and hence they can never be used as recursive parameters of other previously defined functions. Safe composition preserves the distinction between normal and safe arguments by requiring that, when composing along a normal parameter, the pre-composing function has no safe parameter at all. As a result, we can effectively substitute a normal parameter into a safe position but not vice-versa.

Writing **FPTIME** for the class of functions computable in polynomial-time, the main result of Bellantoni and Cook is:

THEOREM 2 [6]. $f(\vec{x}; \cdot) \in B$ if and only if $f(\vec{x}) \in \mathbf{FPTIME}$.

2.2 Proof Theoretic Presentation of Bellantoni–Cook

We shall work with a formulation of Bellantoni and Cook’s algebra as a type system with modalities to distinguish the two sorts (similarly to [22]). In order to facilitate the definition of the circular system that we present later, we, here, work with sequent-style typing derivations.

We only consider *types* (or *formulas*) N (‘safe’) and $\Box N$ (‘normal’) which intuitively vary over the natural numbers. We write A, B , and so on to vary over types.

$$\begin{array}{c}
\text{id} \frac{}{N \Rightarrow N} \quad \text{cut}_N \frac{\Gamma \Rightarrow N \quad \Gamma, N \Rightarrow B}{\Gamma \Rightarrow B} \quad \text{cut}_\square \frac{\Gamma \Rightarrow \square N \quad \square N, \Gamma \Rightarrow B}{\Gamma \Rightarrow B} \quad \text{w}_N \frac{\Gamma \Rightarrow B}{\Gamma, N \Rightarrow B} \quad \text{w}_\square \frac{\Gamma \Rightarrow B}{\square N, \Gamma \Rightarrow B} \quad \text{e} \frac{\Gamma, A, B, \Gamma' \Rightarrow C}{\Gamma, B, A, \Gamma' \Rightarrow C} \\
\\
\text{c}_l \frac{\Gamma, N \Rightarrow A}{\square N, \Gamma \Rightarrow A} \quad \text{c}_r \frac{\square \Gamma \Rightarrow N}{\square \Gamma \Rightarrow \square N} \quad 0 \frac{}{\Rightarrow N} \quad s_0 \frac{\Gamma \Rightarrow A}{\Gamma \Rightarrow A} \quad s_1 \frac{\Gamma \Rightarrow A}{\Gamma \Rightarrow A} \quad \text{srec} \frac{\Gamma \Rightarrow N \quad \square N, \Gamma, N \Rightarrow N \quad \square N, \Gamma, N \Rightarrow N}{\square N, \Gamma \Rightarrow N} \\
\\
\text{cond}_N \frac{\Gamma \Rightarrow N \quad \Gamma, N \Rightarrow N \quad \Gamma, N \Rightarrow N}{\Gamma, N \Rightarrow N} \quad \text{cond}_\square \frac{\Gamma \Rightarrow N \quad \square N, \Gamma \Rightarrow N \quad \square N, \Gamma \Rightarrow N}{\square N, \Gamma \Rightarrow N}
\end{array}$$

Fig. 2. System B, as a sequent-style type system.

A *sequent* is an expression $\Gamma \Rightarrow A$, where Γ is a list of types (called the *context* or *antecedent*) and A is a type (called the *succedent*). For a list of types $\Gamma = N, .^k., N$, we write $\square \Gamma$ for $\square N, .^k., \square N$.

In what follows, we shall essentially identify B with the S4-style type system in Figure 2. The colouring of type occurrences may be ignored for now, they will become relevant in the next section. Derivations in this system are simply called B-derivations, and will be denoted $\mathcal{D}, \mathcal{E}, \dots$. We write $\mathcal{D} : \Gamma \Rightarrow A$ if the derivation $\mathcal{D}$ has end-sequent $\Gamma \Rightarrow A$. We may write $\mathcal{D} = r(\mathcal{D}_1, \dots, \mathcal{D}_n)$ (for $n \leq 3$) if r is the last inference step of $\mathcal{D}$ whose immediate subderivations are, respectively, $\mathcal{D}_1, \dots, \mathcal{D}_n$. Unless otherwise indicated, we assume that the r -instance is as typeset in Figure 2.

Convention 3 (Left Normal, Right Safe). In what follows, we assume that sequents have shape $\square N, \dots, \square N, N, \dots, N \Rightarrow A$, i.e., in the LHS all $\square N$ occurrences are placed before all N occurrences. Note that this invariant is maintained by the typing rules of Figure 2, as long as we insist that $A = B$ in the exchange rule e . This effectively means that exchange steps have one of the following two forms:

$$\begin{array}{c}
\text{e}_N \frac{\Gamma, N, N, \vec{N}' \Rightarrow A}{\Gamma, N, N, \vec{N}' \Rightarrow A} \quad \text{e}_\square \frac{\square \vec{N}, \square N, \square N, \Gamma' \Rightarrow A}{\square \vec{N}, \square N, \square N, \Gamma' \Rightarrow A}
\end{array}$$

Let us point out that this convention does not change the class of definable functions with only normal inputs, under the semantics we are about to give.

Henceforth, we shall often refrain from indicating explicit instances of the exchange rule when constructing derivations.

We construe the system of B-derivations as a class of safe-normal functions by identifying each rule instance as an operation on safe-normal functions. Formally:

Definition 4 (Semantics). Given a B-derivation $\mathcal{D}$ with conclusion $\square N, .^m., \square N, N, .^n., N \Rightarrow A$ we define a two-sorted function $f_{\mathcal{D}}(x_1, \dots, x_m; y_1, \dots, y_n)$ by induction on the structure of $\mathcal{D}$ as follows (all rules as typeset in Figure 2):

- If $\mathcal{D} = \text{id}$ then $f_{\mathcal{D}}(; y) := y$.
- If $\mathcal{D} = \text{w}_N(\mathcal{D}_0)$ then $f_{\mathcal{D}}(\vec{x}; \vec{y}, y) := f_{\mathcal{D}_0}(\vec{x}; \vec{y})$.
- If $\mathcal{D} = \text{w}_\square(\mathcal{D}_0)$ then $f_{\mathcal{D}}(x, \vec{x}; \vec{y}) := f_{\mathcal{D}_0}(\vec{x}; \vec{y})$.
- If $\mathcal{D} = \text{e}_N(\mathcal{D}_0)$ then $f_{\mathcal{D}}(\vec{x}; \vec{y}, y, y', \vec{y}') := f_{\mathcal{D}_0}(\vec{x}; \vec{y}, y', y, \vec{y}')$.
- If $\mathcal{D} = \text{e}_\square(\mathcal{D}_0)$ then $f_{\mathcal{D}}(\vec{x}, x, x', \vec{x}'; \vec{y}) := f_{\mathcal{D}_0}(\vec{x}, x', x, \vec{x}'; \vec{y})$.
- If $\mathcal{D} = \text{c}_l(\mathcal{D}_0)$ then $f_{\mathcal{D}}(x, \vec{x}; \vec{y}) := f_{\mathcal{D}_0}(\vec{x}; \vec{y}, x)$.
- If $\mathcal{D} = \text{c}_r(\mathcal{D}_0)$ then $f_{\mathcal{D}}(\vec{x};) := f_{\mathcal{D}_0}(\vec{x};)$.
- If $\mathcal{D} = 0$ then $f_{\mathcal{D}}(;) := 0$.
- If $\mathcal{D} = s_0(\mathcal{D}_0)$ then $f_{\mathcal{D}}(\vec{x}; \vec{y}) := s_0(; f_{\mathcal{D}_0}(\vec{x}; \vec{y}))$.
- If $\mathcal{D} = s_1(\mathcal{D}_0)$ then $f_{\mathcal{D}}(\vec{x}; \vec{y}) := s_1(; f_{\mathcal{D}_0}(\vec{x}; \vec{y}))$.
- If $\mathcal{D} = \text{cut}_N(\mathcal{D}_0, \mathcal{D}_1)$ then $f_{\mathcal{D}}(\vec{x}; \vec{y}) := f_{\mathcal{D}_1}(\vec{x}; \vec{y}, f_{\mathcal{D}_0}(\vec{x}; \vec{y}))$.

– If $\mathcal{D} = \text{cut}_{\square}(\mathcal{D}_0, \mathcal{D}_1)$ then $f_{\mathcal{D}}(\vec{x}; \vec{y}) := f_{\mathcal{D}_1}(f_{\mathcal{D}_0}(\vec{x}; \vec{y}), \vec{x}; \vec{y})$.

– If $\mathcal{D} = \text{cond}_N(\mathcal{D}_0, \mathcal{D}_1, \mathcal{D}_2)$ then:

$$\begin{aligned} f_{\mathcal{D}}(\vec{x}; \vec{y}, 0) &:= f_{\mathcal{D}_0}(\vec{x}; \vec{y}) \\ f_{\mathcal{D}}(\vec{x}; \vec{y}, s_0 y) &:= f_{\mathcal{D}_1}(\vec{x}; \vec{y}, y) \text{ if } y \neq 0 \\ f_{\mathcal{D}}(\vec{x}; \vec{y}, s_1 y) &:= f_{\mathcal{D}_2}(\vec{x}; \vec{y}, y). \end{aligned}$$

– If $\mathcal{D} = \text{cond}_{\square}(\mathcal{D}_0, \mathcal{D}_1, \mathcal{D}_2)$ then:

$$\begin{aligned} f_{\mathcal{D}}(0, \vec{x}; \vec{y}) &:= f_{\mathcal{D}_0}(\vec{x}; \vec{y}) \\ f_{\mathcal{D}}(s_0 x, \vec{x}; \vec{y}) &:= f_{\mathcal{D}_1}(x, \vec{x}; \vec{y}) \text{ if } x \neq 0 \\ f_{\mathcal{D}}(s_1 x, \vec{x}; \vec{y}) &:= f_{\mathcal{D}_2}(x, \vec{x}; \vec{y}). \end{aligned}$$

– If $\mathcal{D} = \text{srec}(\mathcal{D}_0, \mathcal{D}_1, \mathcal{D}_2)$ then:

$$\begin{aligned} f_{\mathcal{D}}(0, \vec{x}; \vec{y}) &:= f_{\mathcal{D}_0}(\vec{x}; \vec{y}) \\ f_{\mathcal{D}}(s_0 x, \vec{x}; \vec{y}) &:= f_{\mathcal{D}_1}(x, \vec{x}; \vec{y}, f_{\mathcal{D}}(x, \vec{x}; \vec{y})) \text{ if } x \neq 0 \\ f_{\mathcal{D}}(s_1 x, \vec{x}; \vec{y}) &:= f_{\mathcal{D}_2}(x, \vec{x}; \vec{y}, f_{\mathcal{D}}(x, \vec{x}; \vec{y})). \end{aligned}$$

This formal semantics exposes how B-derivations and B functions relate. The rule srec in Figure 2 corresponds to safe recursion, and safe composition along safe parameters is expressed by means of the rules cut_N . Note, however, that the function $f_{\mathcal{D}}$ is not quite defined according to function algebra B, due to the interpretation of the $\text{cut}_{\square}$ rule apparently not satisfying the required constraint on safe composition along a normal parameter. However, this admission turns out to be harmless, as expositied in the following proposition:

PROPOSITION 5. *Given a B-derivation $\mathcal{D} : \square\Gamma, \vec{N} \Rightarrow \square N$, there is a B-derivation $\mathcal{D}^* : \square\Gamma \Rightarrow \square N$ no larger than $\mathcal{D}$ such that, for any $\vec{x}, \vec{y}$:*

$$f_{\mathcal{D}}(\vec{x}; \vec{y}) = f_{\mathcal{D}^*}(\vec{x};).$$

PROOF. The proof is by induction on the size of the derivation. The case where the last rule of $\mathcal{D}$ is an instance of id , 0 , cond_N , $\text{cond}_{\square}$, or srec holds vacuously, while the case $\square_r$ is straightforward. If the last rule of $\mathcal{D}$ is an instance of e_N , $e_{\square}$, $\square_l$, s_i , and $w_{\square}$ then we apply the induction hypothesis. Let us now suppose that $\mathcal{D}$ has been obtained from a derivation $\mathcal{D}_0$ by applying an instance of w_N . By induction hypothesis, there exists a derivation $\mathcal{D}_0^* : \square N, .^{\#}, \square N \Rightarrow \square N$ such that $f_{\mathcal{D}_0}(\vec{x}; \vec{y}) = f_{\mathcal{D}_0^*}(\vec{x};)$. Since $f_{\mathcal{D}}(\vec{x}; \vec{y}, y) = f_{\mathcal{D}_0}(\vec{x}; \vec{y}) = f_{\mathcal{D}_0^*}(\vec{x};)$ we just set $\mathcal{D}^* = \mathcal{D}_0^*$. Suppose now that $\mathcal{D}$ is obtained from two derivations $\mathcal{D}_0$ and $\mathcal{D}_1$ by applying an instance of cut_N . By induction hypothesis, there exists $\mathcal{D}_1^*$ such that $f_{\mathcal{D}_1}(\vec{x}; \vec{y}, y) = f_{\mathcal{D}_1^*}(\vec{x};)$. Since $f_{\mathcal{D}_1}(\vec{x}; \vec{y}, f_{\mathcal{D}_0}(\vec{x}; \vec{y})) = f_{\mathcal{D}_1^*}(\vec{x};)$, we set $\mathcal{D}^* = \mathcal{D}_1^*$. As for the case where the last rule is $\text{cut}_{\square}$, by induction hypothesis, there exist derivations $\mathcal{D}_0^*$ and $\mathcal{D}_1^*$ such that $f_{\mathcal{D}_0}(\vec{x}; \vec{y}) = f_{\mathcal{D}_0^*}(\vec{x};)$ and $f_{\mathcal{D}_1}(\vec{x}, x; \vec{y}) = f_{\mathcal{D}_1^*}(\vec{x}, x;)$, so that we define $\mathcal{D}^*$ as the derivation obtained from $\mathcal{D}_0^*$ and $\mathcal{D}_1^*$ by applying the rule $\text{cut}_{\square}$. $\square$

Our overloading of the notation B, for both a function algebra and for a type system, is now justified by:

PROPOSITION 6. *$f(\vec{x}; \vec{y}) \in \mathbf{B}$ iff there is a B-derivation $\mathcal{D}$ for which $f_{\mathcal{D}}(\vec{x}; \vec{y}) = f(\vec{x}; \vec{y})$.*

PROOF. Let us first prove the left-right implication by induction on $f \in \mathbf{B}$. The cases where f is 0 or s_i are straightforward. If f is a projection then we construct $\mathcal{D}$ using the rules id , w_N , $w_{\square}$, e_N ,

and $e_{\square}$. If $f = p(; x)$ then $\mathcal{D}$ is as follows:

$$\text{cond}_N \frac{0 \xrightarrow{\quad} \Rightarrow N \quad \text{id} \xrightarrow{\quad} N \Rightarrow N \quad \text{id} \xrightarrow{\quad} N \Rightarrow N}{N \Rightarrow N}$$

If f is $\text{cond} (; x, y, z, w)$ then $\mathcal{D}$ is constructed using id , w_N , e_N and cond_N . Suppose now that $f(\vec{x}; \vec{y}) = h(\vec{x}, g(\vec{x};); \vec{y})$. Then $\mathcal{D}$ is as follows:

$$\text{cut}_{\square} \frac{\begin{array}{c} \mathcal{D}_0 \\ \hline \square \vec{N} \Rightarrow N \end{array} \quad \begin{array}{c} \mathcal{D}_1 \\ \hline \square \vec{N}, \square N, \vec{N} \Rightarrow N \end{array}}{\square \vec{N}, \vec{N} \Rightarrow N}$$

where $\mathcal{D}_0$ and $\mathcal{D}_1$ are such that $f_{\mathcal{D}_0} = g$ and $f_{\mathcal{D}_1} = h$. If $f(\vec{x}; \vec{y}) = h(\vec{x}; \vec{y}, g(\vec{x}; \vec{y}))$, then $\mathcal{D}$ is as follows:

$$\text{cut}_N \frac{\begin{array}{c} \mathcal{D}_0 \\ \hline \square \vec{N}, \vec{N} \Rightarrow N \end{array} \quad \begin{array}{c} \mathcal{D}_1 \\ \hline \square \vec{N}, \vec{N}, N \Rightarrow N \end{array}}{\square \vec{N}, \vec{N} \Rightarrow N}$$

where $\mathcal{D}_0$ and $\mathcal{D}_1$ are such that $f_{\mathcal{D}_0} = g$ and $f_{\mathcal{D}_1} = h$. Last, suppose that $f(x, \vec{x}; \vec{y})$ has been obtained by safe recursion from $g(\vec{x}; \vec{y})$ and $h_i(x, \vec{x}; \vec{y}, y)$ with $i = 0, 1$. Then, $\mathcal{D}$ is as follows:

$$\text{srec} \frac{\begin{array}{c} \mathcal{D}_0 \\ \hline \square \vec{N}, \vec{N} \Rightarrow N \end{array} \quad \begin{array}{c} \mathcal{D}_1 \\ \hline \square N, \square \vec{N}, \vec{N}, N \Rightarrow N \end{array} \quad \begin{array}{c} \mathcal{D}_2 \\ \hline \square N, \square \vec{N}, \vec{N}, N \Rightarrow N \end{array}}{\square N, \square \vec{N}, \vec{N} \Rightarrow N}$$

where $\mathcal{D}_0$, $\mathcal{D}_1$, and $\mathcal{D}_2$ are such that $f_{\mathcal{D}_0} = g$, $f_{\mathcal{D}_1} = h_0$ and $f_{\mathcal{D}_2} = h_1$.

For the right-left implication, we prove by induction on the size of derivations that $\mathcal{D} : \square N, .^n, \square N, N, .^m, N \Rightarrow C$ implies $f_{\mathcal{D}} \in B$. The cases where the last rule of $\mathcal{D}$ is id , 0 , s_i , w_N , $w_{\square}$, e_N , $e_{\square}$, $\square_l$, $\square_r$ are all straightforward using constants, successors and projections. If $\mathcal{D}$ has been obtained from two derivations $\mathcal{D}_0$ and $\mathcal{D}_1$ by applying an instance of cut_N then, by applying the induction hypothesis, $f_{\mathcal{D}}(\vec{x}; \vec{y}) = f_{\mathcal{D}_0}(\vec{x}; \vec{y}, f_{\mathcal{D}_1}(\vec{x}; \vec{y})) \in B$. As for the case where the last rule is $\text{cut}_{\square}$, by Proposition 5 there exists a derivation $\mathcal{D}_0^*$ with size no larger than $\mathcal{D}_0$ such that $f_{\mathcal{D}_0}(\vec{x}; \vec{y}) = f_{\mathcal{D}_0^*}(\vec{x};)$. By applying the induction hypothesis, we have $f_{\mathcal{D}}(\vec{x}; \vec{y}) = f_{\mathcal{D}_1}(\vec{x}, f_{\mathcal{D}_0^*}(\vec{x};); \vec{y}) \in B$. If $\mathcal{D}$ has been obtained from derivations $\mathcal{D}_0$, $\mathcal{D}_1$, $\mathcal{D}_2$ by applying an instance of cond_N then, by using the induction hypothesis, $f_{\mathcal{D}}(\vec{x}; \vec{y}, y) = \text{cond} (; y, f_{\mathcal{D}_0}(\vec{x}; \vec{y}), f_{\mathcal{D}_1}(\vec{x}; \vec{y}, p(; y)), f_{\mathcal{D}_2}(\vec{x}; \vec{y}, p(; y))) \in B$. If last rule is $\text{cond}_{\square}$, by induction hypothesis, $f_{\mathcal{D}}(x, \vec{x}; \vec{y}) = \text{cond} (; x, f_{\mathcal{D}_0}(\vec{x}; \vec{y}), f_{\mathcal{D}_1}(p(x;), \vec{x}; \vec{y}), f_{\mathcal{D}_2}(p(x;), \vec{x}; \vec{y}))$, where $p(x;) = p(; \pi_{1;0}^{1;0}(x;))$, and so $f_{\mathcal{D}}(x, \vec{x}; \vec{y}) \in B$. Last, if $\mathcal{D}$ has been obtained from derivations $\mathcal{D}_0$, $\mathcal{D}_1$, $\mathcal{D}_2$ by applying an instance of srec , then $f_{\mathcal{D}} \in B$ using the induction hypothesis and the safe recursion scheme. $\square$

Convention 7. Given Proposition 6 above, we shall henceforth freely write $f(\vec{x}; \vec{y}) \in B$ if there is a derivation $\mathcal{D} : \square \Gamma, \vec{N} \Rightarrow N$ with $f_{\mathcal{D}}(\vec{x}; \vec{y}) = f(\vec{x}; \vec{y})$.

3 Two-Sorted Circular Systems on Notation

In this section, we introduce a ‘coinductive’ version of B , and we study global criteria that tame its computational strength. This proof-theoretic investigation will lead us to two relevant circular systems: CNB, which morally permits ‘nested’ versions of safe recursion, and CB, which will turn out to be closer to usual safe recursion.

Throughout this section we shall work with the set of typing rules $B^- := B - \{\text{srec}\}$.

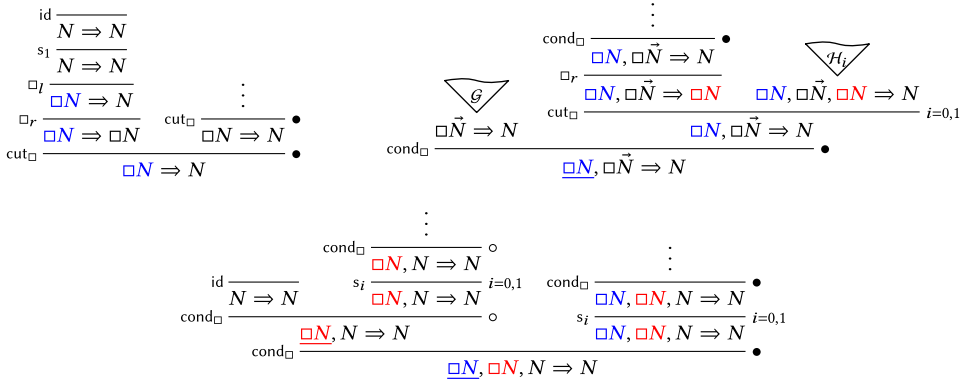


Fig. 3. Examples of regular coderivations $\mathcal{I}$, $\mathcal{R}$ and $\mathcal{C}$, from left (assuming $\mathcal{G}$, $\mathcal{H}_0$ and $\mathcal{H}_1$ regular).

3.1 Non-Wellfounded Typing Derivations

To begin with, we define the notion of ‘coderivation’, which is the fundamental formal object of this section.

Definition 8 (Coderivations). A (B^-) coderivation $\mathcal{D}$ is a possibly infinite rooted tree (of height $\leq \omega$) generated by the rules of B^- . Formally, we identify $\mathcal{D}$ with a prefix-closed subset of $\{0, 1, 2\}^*$ i.e., a ternary tree) where each node is labelled by an inference step from B^- such that, whenever $v \in \mathcal{D}$ is labelled by a step $\frac{s_1 \dots s_n}{s}$, for $n \leq 3$, v has n children in $\mathcal{D}$ labelled by steps with conclusions $S_1, \dots, S_n$, respectively. Sub-coderivations of a coderivation $\mathcal{D}$ rooted at position $v \in \{0, 1, 2\}^*$ are typically denoted $\mathcal{D}_v$, so that $\mathcal{D}_\epsilon = \mathcal{D}$. We write $v \sqsubseteq \mu$ (or $v \sqsubset \mu$) if v is a prefix (respectively, a strict prefix) of μ , and in this case we say that μ is *above* (respectively, *strictly above*) v or that v is *below* (respectively, *strictly below*) μ .

We say that a coderivation is *regular* (or *circular*) if it has only finitely many distinct sub-coderivations.

Note that, while usual derivations may be naturally written as finite trees or dags, regular coderivations may be naturally written as finite directed (possibly cyclic) graphs. Some examples of regular coderivations can be found in Figure 3, employing the following writing conventions:

Convention 9 (Representing coderivations). Henceforth, we may mark steps by $\bullet$ (or similar) in a regular coderivation to indicate roots of identical sub-coderivations. Moreover, to avoid ambiguities and to ease parsing of (co)derivations, we shall often underline principal formulas of a rule instance in a given coderivation and omit instances of $w_\square$ and w_N as well as certain structural steps, e.g., during a cut step.

Finally, when the sub-coderivations $\mathcal{D}_0$ and $\mathcal{D}_1$ above the second and the third premise of the conditional rule (from left) are similar (or identical), we may compress them into a single parametrised sub-coderivation $\mathcal{D}_i$ (for $i = 0, 1$).

As discussed in [13, 15, 25], coderivations can be identified with Kleene–Herbrand–Gödel style equational programs, in general computing partial recursive functionals (see, e.g., [23, Section 63] for further details). We shall specialise this idea to our two-sorted setting.

Definition 10 (Semantics of coderivations). To each B^- -coderivation $\mathcal{D}$ we associate a two-sorted Kleene–Herbrand–Gödel partial function $f_{\mathcal{D}}$ obtained by construing the semantics of Definition 4

as a (possibly infinite) equational program. Given a two-sorted function $f(\vec{x}; \vec{y})$, we say that f is *defined* by a B^- -coderivation $\mathcal{D}$ if $f_{\mathcal{D}}(\vec{x}; \vec{y}) = f(\vec{x}; \vec{y})$.

Remark 11. Note, in particular, that from a regular coderivation $\mathcal{D}$ we obtain a *finite* equational program determining $f_{\mathcal{D}}$. Of course, our overloading of the notation $f_{\mathcal{D}}$ is suggestive since it is consistent with that of Definition 4.

Example 12 (Regular coderivations, revisited). Let us consider the semantics of coderivations $\mathcal{I}$, $\mathcal{R}$ and $\mathcal{C}$ from Figure 3.

— The partial functions $f_{\mathcal{I}_v}$ are given by the following equational program:

$$\begin{aligned} f_{\mathcal{I}_e}(x;) &= f_{\mathcal{I}_1}(f_{\mathcal{I}_0}(x;)) \\ f_{\mathcal{I}_0}(x;) &= f_{\mathcal{I}_{00}}(x;) \\ f_{\mathcal{I}_{00}}(x;) &= f_{\mathcal{I}_{000}}(x;) \\ f_{\mathcal{I}_{000}}(x;) &= s_1(x;) \\ f_{\mathcal{I}_1}(x;) &= f_{\mathcal{I}_e}(x;). \end{aligned}$$

By purely equational reasoning, we can simplify this program to obtain $f_{\mathcal{I}_e}(x;) = f_{\mathcal{I}_e}(s_1 x;)$. Since the above equational program keeps increasing the input, the function $f_{\mathcal{I}} = f_{\mathcal{I}_e}$ is always undefined.

— Let $f_{\mathcal{G}}(\vec{x}; \vec{y})$ and $f_{\mathcal{H}_i}(x, \vec{x}, z; \vec{y})$ ($i = 0, 1$) be the functions defined by the regular B^- -coderivations $\mathcal{G}$ and $\mathcal{H}_i$, respectively. Then the equational program for $\mathcal{R}$ can be rewritten as follows:

$$\begin{aligned} f_{\mathcal{R}}(0, \vec{x};) &= f_{\mathcal{G}}(\vec{x};) \\ f_{\mathcal{R}}(s_i x, \vec{x};) &= f_{\mathcal{H}_i}(x, \vec{x}, f_{\mathcal{R}}(x, \vec{x};);), \end{aligned} \tag{1}$$

which is an instance of a *non-safe* recursion scheme (on notation).

— The equational program of $\mathcal{C}$ can be written as:

$$\begin{aligned} f_{\mathcal{C}_e}(0, 0; z) &= z \\ f_{\mathcal{C}_e}(0, s_i y; z) &= s_i f_{\mathcal{C}_e}(x, y; z) \neq 0 \\ f_{\mathcal{C}_e}(s_i x, y; z) &= s_i f_{\mathcal{C}_e}(x, y; z) \neq 0, \end{aligned}$$

which computes concatenation of the binary representation of three natural numbers.

The above examples illustrate two undesirable features of regular B^- -coderivations, from the point of view of implicit complexity:

- I. On the one hand, despite being finitely presentable, they can define *partial functions*;
- II. On the other hand, despite the presence of modalities implementing the normal/safe distinction of function arguments, they can define *non-safe recursion schemes*.

3.2 The Progressing Criterion

To address Problem 3.1 we shall adapt to our setting a well-known ‘termination criterion’ from non-wellfounded proof theory. First, let us recall some standard proof theoretic concepts about (co)derivations, similar to those in [13, 15, 25].

Definition 13 (Ancestry). Fix a coderivation $\mathcal{D}$. We say that a type occurrence A is an *immediate ancestor* of a type occurrence B in $\mathcal{D}$ if they are types in a premiss and conclusion (respectively) of an inference step and, as typeset in Figure 2, have the same colour. If A and B are in some Γ or Γ' , then furthermore they must be in the same position in the list.

Being a binary relation, immediate ancestry forms a directed graph upon which our correctness criterion is built.

Definition 14 (Progressing coderivations). A *thread* is a maximal path in the graph of immediate ancestry. We say that a (infinite) thread is *progressing* if it is eventually constant $\Box N$ and infinitely often principal for a $\text{cond}_{\Box}$ rule.

A coderivation is *progressing* if each of its infinite branches has a progressing thread.

Example 15 (Regular coderivations, re-revisited). In Figure 3, $\mathcal{I}$ has precisely one infinite branch (that loops on $\bullet$) which contains no instances of $\text{cond}_{\Box}$ at all. Therefore, $\mathcal{I}$ is not progressing. On the other hand, $\mathcal{C}$ has two simple loops, one on $\bullet$ and the other one on $\circ$. For any infinite branch B we have two cases:

- if B crosses the bottommost conditional infinitely many times, it contains a progressing blue thread;
- otherwise, B crosses the topmost conditional infinitely many times, so that it contains a progressing red thread.

Therefore, $\mathcal{C}$ is progressing. By similar reasoning, we can conclude that $\mathcal{R}$ is progressing whenever $\mathcal{G}$ and $\mathcal{H}_i$ are.

Like in [13, 15, 25], the progressing criterion is sufficient to guarantee that the partial function computed is, in fact, a well-defined total function:

PROPOSITION 16. *If $\mathcal{D}$ is progressing then $f_{\mathcal{D}}$ is total.*

PROOF SKETCH. We proceed by contradiction. If $f_{\mathcal{D}}$ is non-total then, since each rule preserves totality top-down, we must have that $f_{\mathcal{D}'}$ is non-total for one of $\mathcal{D}$'s immediate sub-coderivations $\mathcal{D}'$. Continuing this reasoning we can build an infinite leftmost 'non-total' branch $B = (\mathcal{D}^i)_{i < \omega}$. Let $(\Box N^i)_{i \geq k}$ be a progressing thread along B , and assign to each $\Box N^i$ the least natural number $n_i \in \mathbb{N}$ such that $f_{\mathcal{D}'}$ is non-total when n_i is assigned to the type occurrence $\Box N^i$.

Now, notice that:

- $(n_i)_{i \geq k}$ is monotone non-increasing, by inspection of the rules and their interpretations from Definition 4.
- $(n_i)_{i \geq k}$ does not converge, since $(\Box N^i)_{i \geq k}$ is progressing and so is infinitely often principal for $\text{cond}_{\Box}$, where the value of n_i must strictly decrease (cf., again, Definition 4).

This contradicts the well-ordering property of the natural numbers. $\square$

One of the most appealing features of the progressing criterion is that, while being rather expressive and admitting many natural programs, e.g., as we will see in the next subsections, it remains effective (for regular coderivations) thanks to well-known arguments in automaton theory:

FACT 17 (FOLKLORE). It is decidable whether a regular coderivation is progressing.

This well-known result (see, e.g., [18] for an exposition for a similar circular system) follows from the fact that the progressing criterion is equivalent to the universality of a Büchi automaton of size determined by the (finite) representation of the input coderivation. This problem is decidable in polynomial space, though the correctness of this algorithm requires nontrivial infinitary combinatorics, as formally demonstrated in [24].

Let us finally observe that the progressing condition turns out to be sufficient to restate Proposition 5 in the setting of non-wellfounded coderivations:

PROPOSITION 18. *Given a progressing B^- -coderivation $\mathcal{D} : \Box\Gamma, \vec{N} \Rightarrow \Box N$, there is a progressing B^- -coderivation $\mathcal{D}^* : \Box\Gamma \Rightarrow \Box N$ such that, for any $\vec{x}, \vec{y}$:*

$$f_{\mathcal{D}}(\vec{x}; \vec{y}) = f_{\mathcal{D}^*}(\vec{x};).$$

PROOF. By progressiveness, any infinite branch contains a $\text{cond}_{\Box}$ -step, which has non-modal succedent. Thus, there is a set of $(\text{cond}_{\Box}, 0, \text{id})$ -occurrences that forms a bar across $\mathcal{D}$. By König Lemma, the set of all nodes of $\mathcal{D}$ below this bar, say $X_{\mathcal{D}}$, is finite. The proof now follows by induction on the size of $X_{\mathcal{D}}$, and proceeds analogously to Proposition 5. If the size is 0 then the last rule of $\mathcal{D}$ must be an instance of $\text{cond}_{\Box}, 0$, or id , in which case the statement holds vacuously. Otherwise, we do case analysis. If the last rule of $\mathcal{D}$ is an instance of $\Box_r$ or cond_N then the statement holds vacuously. If the last rule of $\mathcal{D}$ is an instance of $e_N, e_{\Box}, \Box_l, s_i$, and $w_{\Box}$ then we apply the induction hypothesis. Let us now suppose that $\mathcal{D}$ has been obtained from a coderivation $\mathcal{D}_0$ by applying an instance of w_N . By induction hypothesis, there exists a coderivation $\mathcal{D}_0^* : \Box N, .^n., \Box N \Rightarrow \Box N$ such that $f_{\mathcal{D}_0}(\vec{x}; \vec{y}) = f_{\mathcal{D}_0^*}(\vec{x};)$. Since $f_{\mathcal{D}}(\vec{x}; \vec{y}, y) = f_{\mathcal{D}_0}(\vec{x}; \vec{y}) = f_{\mathcal{D}_0^*}(\vec{x};)$ we just set $\mathcal{D}^* = \mathcal{D}_0^*$. Suppose now that $\mathcal{D}$ is obtained from two coderivations $\mathcal{D}_0$ and $\mathcal{D}_1$ by applying an instance of cut_N . By induction hypothesis, there exists $\mathcal{D}_1^*$ such that $f_{\mathcal{D}_1}(\vec{x}; \vec{y}, y) = f_{\mathcal{D}_1^*}(\vec{x};)$. Since $f_{\mathcal{D}_1}(\vec{x}; \vec{y}, f_{\mathcal{D}_0}(\vec{x}; \vec{y})) = f_{\mathcal{D}_1^*}(\vec{x};)$, we set $\mathcal{D}^* = \mathcal{D}_1^*$. As for the case where the last rule is $\text{cut}_{\Box}$, by induction hypothesis, there exist coderivations $\mathcal{D}_0^*$ and $\mathcal{D}_1^*$ such that $f_{\mathcal{D}_0}(\vec{x}; \vec{y}) = f_{\mathcal{D}_0^*}(\vec{x};)$ and $f_{\mathcal{D}_1}(\vec{x}, x; \vec{y}) = f_{\mathcal{D}_1^*}(\vec{x}, x;)$, so that we define $\mathcal{D}^*$ as the coderivation obtained from $\mathcal{D}_0^*$ and $\mathcal{D}_1^*$ by applying the rule $\text{cut}_{\Box}$. $\square$

Note that the above proof only depends on progressiveness, and so it holds for non-regular progressing B^- -coderivations as well.

3.3 Computational Expressivity of Coderivations

Problem II indicates that the modal/non-modal distinction for (progressing) B^- -coderivations is somewhat redundant and does not suffice, by itself, to control complexity. Indeed, by inspection of Figure 2 it is not hard to see that we can stepwise replace each N by $\Box N$ in the coderivation while preserving progressiveness, so that all definable functions with normal and safe inputs can be defined by using only normal inputs.

PROPOSITION 19. *Let $\mathcal{D} : \Box N, .^n., \Box N, N, .^m., N \Rightarrow N$ be a B^- -coderivation. Then, there exists a B^- -coderivation $\mathcal{D}^{\Box} : \Box N, .^{n+m}, \Box N \Rightarrow N$ s.t.:*

- $f_{\mathcal{D}}(\vec{x}; \vec{y}) = f_{\mathcal{D}^{\Box}}(\vec{x}, \vec{y};)$;
- $\mathcal{D}^{\Box}$ does not contain instances of $w_N, e_N, \text{cut}_N, \text{cond}_N$.

Moreover, $\mathcal{D}^{\Box}$ is regular (resp. progressing) if $\mathcal{D}$ is.

PROOF. We construct $\mathcal{D}^{\Box}$ coinductively. We only consider two interesting cases, when $\mathcal{D}$ is id or it is obtained from $\mathcal{D}_0 : \Gamma \Rightarrow N$ and $\mathcal{D}_1 : \Gamma, N \Rightarrow A$ by applying a cut_N -step. Then, $\mathcal{D}^{\Box}$ is constructed, respectively, as follows:

$$\begin{array}{c}
 \text{id} \text{ ————— } \\
 \Box_l \frac{N \Rightarrow N}{\Box N \Rightarrow N}
 \end{array}
 \qquad
 \begin{array}{c}
 \text{cut}_{\Box} \frac{
 \begin{array}{c}
 \text{---} \triangle_{\mathcal{D}_0^{\Box}} \text{---} \\
 \Box_r \frac{\Box \Gamma \Rightarrow N}{\Box \Gamma \Rightarrow \Box N}
 \end{array}
 \quad
 \begin{array}{c}
 \text{---} \triangle_{\mathcal{D}_1^{\Box}} \text{---} \\
 \Box_l \frac{\Box \Gamma, N \Rightarrow N}{\Box \Gamma, \Box N \Rightarrow N}
 \end{array}
 }{
 \Box \Gamma \Rightarrow N
 }
 \end{array}$$

the remaining interesting cases are treated similarly. $\square$

Consequently, we may view (regular, progressing) B^- -coderivations as the type 0 (regular, progressing) fragment of the system CT from [13, 15, 25].

As a result we inherit the following characterisations:

PROPOSITION 20. *We have the following:*

- (1) *Any function $f(\vec{x};)$ is defined by a progressing B^- -coderivation.*
- (2) *The class of regular B^- -coderivations is Turing-complete, i.e. they define every partial recursive function.*
- (3) *$f(\vec{x};)$ is defined by a regular progressing B^- -coderivation if and only if $f(\vec{x})$ is type-1-primitive-recursive, i.e., it is in the level 1 fragment T_1 of Gödel's T .*

The proof of the above proposition can be found in Appendix A.

Given the computationally equivalent system CT_0 with contraction from [15], we can view the above result as a sort of ‘contraction admissibility’ for regular progressing B^- -coderivations. Call $B^- + \{c_N, c_{\Box N}\}$ the extension of B^- with the rules c_N and $c_{\Box N}$ below:

$$c_N \frac{\Gamma, N, N \Rightarrow B}{\Gamma, N \Rightarrow B} \quad c_{\Box} \frac{\Box N, \Box N, \Gamma \Rightarrow B}{\Box N, \Gamma \Rightarrow B}, \quad (2)$$

where the semantics for the new system extends the one for B^- in the obvious way, and the notion of (progressing) thread is induced by the given colouring.¹ We have:

COROLLARY 21. *$f(\vec{x};)$ is definable by a regular progressing $B^- + \{c_N, c_{\Box N}\}$ -coderivation if and only if it is definable by a regular progressing B^- -coderivation.*

3.4 Proof-Level Conditions Motivated by Implicit Complexity

B -derivations locally introduce safe recursion by means of the rule $srec$, and Proposition 5 ensures that the composition schemes defined by the cut rules are safe. As suggested by Problem 3.1, a different situation arises when we move to B^- -coderivations, where the lack of further constraints means that we can define ‘non-safe’ equational programs. We may recover safety by a natural proof-level condition:

Definition 22 (Safety). A B^- -coderivation is *safe* if each infinite branch crosses only finitely many $cut_{\Box}$ -steps.

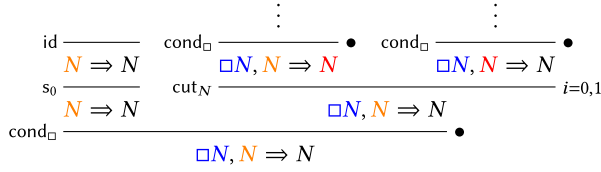
The corresponding equational programs of safe coderivations indeed only have safe inputs in hereditarily safe positions, as we shall soon see. Let us illustrate this by means of examples.

Example 23. The coderivation $\mathcal{I}$ in Figure 3 is not safe, as there is an instance of $cut_{\Box}$ in the loop on $\bullet$, which means that there is an infinite branch crossing infinitely many $cut_{\Box}$ -steps. By contrast, the coderivation $\mathcal{C}$ is safe because it does not contain instances of the rule $cut_{\Box}$. Finally, by inspecting the coderivation $\mathcal{R}$ of Figure 3, we notice that the infinite branch that loops on $\bullet$ contains infinitely many $cut_{\Box}$ steps, so it is not safe.

Perhaps surprisingly, however, the safety condition is not enough to restrict the set of B^- -definable functions to **FPTIME**, as the following example shows.

¹Note that the totality argument of Proposition 16 still applies in the presence of these rules, cf. also [15].

Example 24 (Safe exponentiation). Consider the following coderivation $\mathcal{E}$,



where we identify the sub-coderivations above the second and the third premises of the conditional. The coderivation is clearly progressing. Moreover it is safe, as $\mathcal{E}$ has no instances of $\text{cut}_\square$. Its associated equational program can be written as follows:

$$\begin{aligned} f_{\mathcal{E}}(0; y) &= s_0(; y) \\ f_{\mathcal{E}}(s_0 x; y) &= f_{\mathcal{E}}(x; f_{\mathcal{E}}(x; y)) \quad x \neq 0 \\ f_{\mathcal{E}}(s_1 x; y) &= f_{\mathcal{E}}(x; f_{\mathcal{E}}(x; y)). \end{aligned} \quad (3)$$

The above equational program has already appeared in [22, 27]. It is not hard to show, by induction on x , that $f_{\mathcal{E}}(x; y) = 2^{2^{|x|}} \cdot y$. Thus, $f_{\mathcal{E}}$ has exponential growth rate (as long as $y \neq 0$), despite being defined by a ‘safe’ recursion scheme.

The above coderivation exemplifies a safe recursion scheme that is able to *nest* one recursive call inside another in order to obtain exponential growth rate. This is in fact a peculiar feature of circular proofs, and it is worth discussing.

Remark 25 (On nesting and higher-order recursion). As we have seen, namely in Proposition 20.3, (progressing) B^- -coderivations are able to simulate some sort of higher-order recursion, namely at type 1 (cf. also [15]). In this way it is arguably not so surprising that the sort of ‘nested recursion’ in Equation (3) is definable since type 1 recursion, in particular, allows such nesting of the recursive calls. To make this point more apparent, consider the following higher-order ‘safe’ recursion operator:

$$\text{rec}_A : \square N \rightarrow (\square N \rightarrow A \rightarrow A) \rightarrow A \rightarrow A,$$

with $A = N \rightarrow N$, and $f(x) = \text{rec}_A(x, h, g)$ is defined as $f(0) = g$ and $f(s_i x) = h(x, f(x))$ for $x > 0$. By setting $g := \lambda y : N. s_0 y$ and $h := \lambda x : \square N. \lambda u : N \rightarrow N. \lambda y : N. u(u y)$ we can easily check that $f_{\mathcal{E}}(x; y) = \text{rec}_A(x, h, g)(y)$, where $\mathcal{E}$ is as in Example 24. Hence, the function $f_{\mathcal{E}}(x; y)$ can be defined by means of a higher-order version of safe recursion.

As noticed by Hofmann [22], and formally proved by Leivant [27], **FELEMENTARY** can be characterised using higher-order safe recursion, thanks to this capacity to nest recursive calls. Moreover, Hofmann showed in [22] that by introducing a ‘linearity’ restriction on the operator rec_A , which prevents duplication of recursive calls, it is possible to recover the class **FPTIME**. The resulting type system, called SLR (‘Safe Linear Recursion’), can thus be regarded as a higher-order formulation of B .

Following [22], we shall impose a linearity criterion to rule out those coderivations that nest recursive calls. This is achieved by observing that the duplication of the recursive calls of $f_{\mathcal{E}}$ in Example 24 is due to the presence in $\mathcal{E}$ of loops on $\bullet$ crossing both premises of a cut_N step. Hence, our circular-proof-theoretic counterpart of Hofmann’s linearity restriction can be obtained by demanding that only the left premiss of each cut_N step is crossed by such loops. Again, we rather give a more natural proof-level criterion which does not depend on our intuitive notion of loop.

Definition 26 (Left-leaning). A B^- -coderivation is said to be *left-leaning* if each infinite branch goes right at a cut_N -step only finitely often.

Example 27. In Figure 3, $\mathcal{I}$ is trivially left-leaning, as it contains no instances of cut_N at all. The coderivations $\mathcal{C}$ and $\mathcal{R}$ are also left-leaning, since no infinite branch can go right at the cut_N steps. By contrast, the coderivation $\mathcal{E}$ in Example 24 is not left-leaning, as there is an infinite branch looping at $\bullet$ and crossing infinitely many times the rightmost premise of the cut_N -step.

We are now ready to present our circular systems:

Definition 28 (Circular Implicit Systems). CNB is the class of safe regular progressing B^- -coderivations. CB is the restriction of CNB to only left-leaning coderivations. A two-sorted function $f(\vec{x}; \vec{y})$ is CNB-definable (or CB-definable) if there is a coderivation $\mathcal{D} \in \text{CNB}$ (resp., $\mathcal{D} \in \text{CB}$) such that $f_{\mathcal{D}}(\vec{x}; \vec{y}) = f(\vec{x}; \vec{y})$.

Let us point out that Proposition 18 can be strengthened to preserve safety and left-leaningness:

PROPOSITION 29. Let $\mathcal{D} : \Box\Gamma, \vec{N} \Rightarrow \Box N$ be a coderivation in CNB (or CB). There exists a CNB-coderivation (resp., CB-coderivation) $\mathcal{D}^* : \Box\Gamma \Rightarrow \Box N$ such that $f_{\mathcal{D}}(\vec{x}; \vec{y}) = f_{\mathcal{D}^*}(\vec{x}; \vec{y})$.

3.5 On the Complexity of Proof-Checking

Note that both the safety and the left-leaning conditions above are defined at the level of arbitrary coderivations, not just regular and/or progressing ones. Moreover, since these conditions are defined at the proof-level rather than the thread-level, they are easy to check on regular coderivations:

PROPOSITION 30. The safety and the left-leaning condition are NL-decidable for regular coderivations.

PROOF. We can represent a regular coderivation $\mathcal{D}$ as a finite directed (possibly cyclic) graph $G_{\mathcal{D}}$ labelled with inference rules. Then, the problem of deciding whether $\mathcal{D}$ is not safe (resp. left leaning) comes down to the problem of deciding whether a cycle π of $G_{\mathcal{D}}$ exists crossing a node labelled $\text{cut}_{\Box}$ (resp. crossing the rightmost child of a node labelled cut_N). W.l.o.g. we can assume that π is a simple cycle. Given (an encoding of) $G_{\mathcal{D}}$ as read-only input and (an encoding of) π as a read-only certificate, we can easily construct a deterministic Turing machine M verifying that the cycle π crosses $\text{cut}_{\Box}$ (resp. the rightmost child of a node cut_N) in $G_{\mathcal{D}}$. More specifically:

- the size of the certificate is smaller than the size of the description of $G_{\mathcal{D}}$;
- M reads once from left to right the addresses of the certificate which provide the information about where to move the pointers;
- M works in logspace as it only stores addresses in memory. □

The idea here is that, for regular coderivations, checking that no branch has infinitely many occurrences of a particular rule can be reduced to checking acyclicity of a certain subgraph, which is well-known to be in $\text{coNL} = \text{NL}$.

Recall that progressiveness of regular coderivations is decidable by reduction to universality of Büchi automata, a PSPACE-complete problem. Indeed progressiveness itself is PSPACE-complete in many settings, cf. [31]. It is perhaps surprising, therefore, that the safety of a regular coderivation also allows us to decide progressiveness efficiently too, thanks to the following reduction:

PROPOSITION 31. A safe B^- -coderivation is progressing iff every infinite branch has infinitely many $\text{cond}_{\Box}$ -steps.

PROOF. The left-right implication is trivial. For the right-left implication, let us consider an infinite branch B of a safe B^- -coderivation $\mathcal{D}$. By safety, there exists a node v of B such that any sequent

<i>safe recursion</i>	<i>on notation</i>	<i>on $\subset$</i>
<i>unnested</i>	B	$B^\subset$
<i>nested</i>	NB	$NB^\subset$

Fig. 4. The function algebras considered in Section 4. Any algebra is included in one below it and to the right of it.

above ν in B is not the conclusion of a $\text{cut}_\square$ -step. Now, by inspecting the rules of $B^- - \{\text{cut}_\square\}$ we observe that:

- every modal formula occurrence in B has a unique thread along B ;
- infinite threads along B cannot start strictly above ν .

Hence, setting k to be the number of $\square N$ occurrences in the antecedent of ν , B has (at most) k infinite threads. Moreover, since B contains infinitely many $\text{cond}_\square$ -steps, by the Infinite Pigeonhole Principle we conclude that one of these threads is infinitely often principal for the $\text{cond}_\square$ rule. $\square$

Thus, using similar reasoning to that of Proposition 30 we may conclude from Proposition 31 the following:

COROLLARY 32. *Given a regular B^- -coderivation $\mathcal{D}$, the problem of deciding if $\mathcal{D}$ is in CNB (resp. CB) is in NL.*

Let us point out that the reduction above is similar to (and indeed generalises) an analogous one for cut-free extensions of Kleene algebra, cf. [17, Proposition 8].

4 Some Variants of Safe Recursion

In this section, we shall introduce various extensions of B to ultimately classify the expressivity of the circular systems CB and CNB. First, starting from the analysis of Example 24 and the subsequent system CNB, we shall define a version of B with safe *nested* recursion, called NB. Second, motivated by the more liberal way of defining functions in both CB and CNB, we shall endow the function algebras B and NB with forms of safe recursion over a well-founded relation $\subset$ on lists of normal parameters. Figure 4 summarises the function algebras considered and their relations.

4.1 Relativised Algebras and Nested Recursion

One of the key features of the Bellantoni–Cook algebra B is that ‘nesting’ of recursive calls is not permitted. For instance, let us recall the equational program from Example 24:

$$\begin{aligned}
 \text{ex}(0; y) &= s_0 y \\
 \text{ex}(s_0 x; y) &= \text{ex}(x; \text{ex}(x; y)) \quad x \neq 0 \\
 \text{ex}(s_1 x; y) &= \text{ex}(x; \text{ex}(x; y)).
 \end{aligned} \tag{4}$$

Recall that $\text{ex}(x; y) = f_{\mathcal{E}}(x; y) = 2^{2^{|x|}} \cdot y$. The ‘recursion step’ on the second line is compatible with safe composition, in that safe inputs only occur in hereditarily safe positions, but one of the recursive calls takes another recursive call among its safe inputs. In Example 24, we showed how the above function $\text{ex}(x; y)$ can be CNB-defined. We thus seek a suitable extension of B able to formalise such nested recursion to serve as a function algebraic counterpart to CNB.

It will be convenient for us to work with generalisations of B including *oracles*.

Definition 33. For all sets of oracles $\vec{a} = a_1, \dots, a_k$, we define the algebra of B^- functions *over $\vec{a}$* to include all the initial functions of B^- and

- (oracles). $a_i(\vec{x}; \vec{y})$ is a function over $\vec{a}$, for $1 \leq i \leq k$ (where $\vec{x}, \vec{y}$ have appropriate length for a_i). and closed under:
- (Safe Composition).
 - (1) from $g(\vec{x}; \vec{y}), h(\vec{x}; \vec{y}, y)$ over $\vec{a}$ define $f(\vec{x}; \vec{y})$ over $\vec{a}$ by $f(\vec{x}; \vec{y}) = h(\vec{x}; \vec{y}, g(\vec{x}; \vec{y}))$.
 - (2) from $g(\vec{x};)$ over $\emptyset$ and $h(\vec{x}, x; \vec{y})$ over $\emptyset$ define $f(\vec{x}; \vec{y})$ over $\vec{a}$ by $f(\vec{x}; \vec{y}) = h(\vec{x}, g(\vec{x};); \vec{y})$.

We write $B^-(\vec{a})$ for the class of functions over $\vec{a}$ generated in this way.

Henceforth, we may write $f(\vec{a})(\vec{x}; \vec{y})$ to stress that the function $f(\vec{x}; \vec{y})$ is over oracles $\vec{a}$.

Remark 34. Formally, a function $f(\vec{a})(\vec{x}; \vec{y})$ can be understood as a type 2 functional that takes type 1 inputs $\vec{a}$ (safe-normal functions), and type 0 inputs $\vec{x}$ (normal) and $\vec{y}$ (safe). With little abuse of notation, we may represent this functional as $\lambda \vec{a}. f(\vec{a})(\vec{x}; \vec{y})$, where $\vec{a}$ are now *variables* ranging over first-order functions (with appropriate sorts). Notice that this justifies replacing an oracle with another first-order function (with matching sorts). However, to avoid any reference to higher-order functions, in this article we prefer to adopt the parametric notation $f(\vec{a})(\vec{x}; \vec{y})$ where oracles are arbitrary but fixed *functions*.

Note that Safe Composition along normal parameters (Item above) comes with the condition that $g(\vec{x};)$ is oracle-free. This restriction prevents oracles appearing in normal position. Since our recursion schemes replace oracles with recursive calls, the latter will also be required to appear in safe position. The same condition on $h(\vec{x}, x; \vec{y})$ being oracle-free is not strictly necessary for the complexity bounds we are after, as we shall see in the next section when we define more expressive algebras, but is convenient in order to facilitate the ‘grand tour’ strategy of this article (cf. Figure 1).

We shall write, say, $\lambda \vec{v}. f(\vec{x}; \vec{v})$ for the function taking only safe arguments $\vec{v}$ with $(\lambda \vec{v}. f(\vec{x}; \vec{v}))(\vec{y}) = f(\vec{x}; \vec{y})$ (here $\vec{x}$ may be seen as parameters). Nested recursion can be formalised in the setting of algebras-with-oracles as follows:

Definition 35 (Safe Nested Recursion). We write snrec for the scheme:

- from $g(\vec{x}; \vec{y})$ over $\emptyset$ and $h_i(a)(x, \vec{x}; \vec{y})$ over $a, \vec{a}$, define $f(x, \vec{x}; \vec{y})$ over $\vec{a}$ by:

$$\begin{aligned} f(0, \vec{x}; \vec{y}) &= g(\vec{x}; \vec{y}) \\ f(s_0 x, \vec{x}; \vec{y}) &= h_0(\lambda \vec{v}. f(x, \vec{x}; \vec{v}))(x, \vec{x}; \vec{y}) \quad x \neq 0 \\ f(s_1 x, \vec{x}; \vec{y}) &= h_1(\lambda \vec{v}. f(x, \vec{x}; \vec{v}))(x, \vec{x}; \vec{y}). \end{aligned}$$

We write $NB(\vec{a})$ for the class of functions over $\vec{a}$ generated from B^- under snrec and Safe Composition (from Definition 33), and write simply NB for $NB(\emptyset)$.

Remark 36 (Safe Composition During Safe Recursion). Note that Safe Nested Recursion also admits variants that are not morally ‘nested’ but rather use a form of ‘composition during recursion’:

- from $g(\vec{x}; \vec{y}), (\vec{g}_j(x, \vec{x}; \vec{y}))_{1 \leq j \leq k}$ and $h_i(x, \vec{x}; \vec{y}, (z_j)_{1 \leq j \leq k})$ define:

$$\begin{aligned} f(0, \vec{x}; \vec{y}) &= g(\vec{x}; \vec{y}) \\ f(s_i x, \vec{x}; \vec{y}) &= h_i \left(x, \vec{x}; \vec{y}, (f(x, \vec{x}; \vec{g}_j(x, \vec{x}; \vec{y})))_{1 \leq j \leq k} \right). \end{aligned} \tag{5}$$

In this case, note that we have allowed the safe inputs of f to take arbitrary values given by previously defined functions, but at the same time f never calls itself in a safe position, as in (4). In the conference version of this article [11, p. 8] we made an unsubstantiated claim that a function algebra extending B^- by (5) above is equivalent to a restriction SB of NB requiring that oracles are *unnested* in Safe Composition: in item, one of $g(\vec{x}; \vec{y})$ and $h(\vec{x}; \vec{y}, y)$ must be oracle-free, i.e.,

over $\emptyset$. Such an equivalence is in fact not immediate, as our Safe Composition scheme allows substitution only along a single parameter, so there does not seem to be a direct way of expressing Equation (5) in NB without nesting oracles, in the sense just described. While we did not rely on this equivalence for any of the main results of [11], in this work we simply omit reference to SB and its variants to avoid any confusion.

4.2 Safe Recursion on Well-Founded Relations

Relativised function algebras may be readily extended by recursion on arbitrary well-founded relations. For instance, given a well-founded preorder $\trianglelefteq$, and writing $\triangleleft$ for its strict variant,² ‘safe recursion on $\triangleleft$ ’ is given by the scheme:

— from $h(a)(x, \vec{x}; \vec{y})$ over $a, \vec{a}$, define $f(x, \vec{x}; \vec{y})$ over $\vec{a}$ by:

$$f(x, \vec{x}; \vec{y}) = h(\lambda v \triangleleft x. f(v, \vec{x}; \vec{y}))(x, \vec{x}; \vec{y}).$$

Note here that we employ the notation $\lambda v \triangleleft x$ for a ‘guarded abstraction’. Formally:

$$(\lambda v \triangleleft x. f(v, \vec{x}; \vec{y}))(z) := \begin{cases} f(z, \vec{x}; \vec{y}) & z \triangleleft x \\ 0 & \text{otherwise} \end{cases}$$

It is now not hard to see that total functions (with oracles) are closed under the recursion scheme above, by reduction to induction on the well-founded relation $\triangleleft$.

Note that such schemes can be naturally extended to preorders on *tuples* of numbers too, by abstracting several inputs. We shall specialise this idea to a particular well-founded preorder that will be helpful later to bound the complexity of definable functions in our systems CB and CNB.

Recall that we say that x is a *prefix* of y if y has the form xz in binary notation, i.e., y can be written $x2^n + z$ for some $n \geq 0$ and some $z < 2^n$. We say that x is a *strict prefix* of y if x is a prefix of y but $x \neq y$.

Definition 37 (Permutations of prefixes). Let $[n]$ denote $\{0, \dots, n-1\}$. We write $(x_0, \dots, x_{n-1}) \subseteq (y_0, \dots, y_{n-1})$ if, for some permutation $\pi : [n] \rightarrow [n]$, we have that x_i is a prefix of $y_{\pi i}$, for all $i < n$. We write $\vec{x} \subseteq \vec{y}$ if $\vec{x} \subseteq \vec{y}$ but $\vec{y} \not\subseteq \vec{x}$, i.e., there is a permutation $\pi : [n] \rightarrow [n]$ with x_i a prefix of y_i for each $i < n$ and, for some $i < n$, x_i is a strict prefix of y_i .

It is not hard to see that $\subseteq$ is a well-founded preorder, by reduction to the fact that the prefix relation is a well-founded partial order. As a result, we may duly devise a version of safe (nested) recursion on $\subseteq$:

Definition 38 (Safe (nested) recursion on permutations of prefixes). We write $\text{NB}^\subseteq(\vec{a})$ for the class of functions over $\vec{a}$ generated from B^- under Safe Composition, the scheme $\text{snrec}_\subseteq$,

— from $h(a)(\vec{x}; \vec{y})$ over $a, \vec{a}$ define $f(\vec{x}; \vec{y})$ over $\vec{a}$ by:

$$f(\vec{x}; \vec{y}) = h(\lambda \vec{u} \subseteq \vec{x}. \lambda \vec{v} \subseteq \vec{y}. f(\vec{u}; \vec{v}))(\vec{x}; \vec{y}),$$

and the following generalisation of Safe Composition along a Normal Parameter:

(2)′ from $g(\vec{x};)$ over $\emptyset$ and $h(\vec{x}, x; \vec{y})$ over $\vec{a}$ define $f(\vec{x}; \vec{y})$ over $\vec{a}$ by $f(\vec{x}; \vec{y}) = h(\vec{x}, g(\vec{x};); \vec{y})$.

We define $\text{B}^\subseteq(\vec{a})$ to be the restriction of $\text{NB}^\subseteq(\vec{a})$ where every instance of $\text{snrec}_\subseteq$ has the form:

— from $h(a)(\vec{x}; \vec{y})$ over $a, \vec{a}$, define $f(\vec{x}; \vec{y})$ over $\vec{a}$:

$$f(\vec{x}; \vec{y}) = h(\lambda \vec{u} \subseteq \vec{x}. \lambda \vec{v} \subseteq \vec{y}. f(\vec{u}; \vec{v}))(\vec{x}; \vec{y}).$$

²To be precise, for a preorder $\trianglelefteq$ we write $x \triangleleft y$ if $x \trianglelefteq y$ and $y \not\trianglelefteq x$. As abuse of terminology, we say that $\trianglelefteq$ is well-founded just when $\triangleleft$ is.

We call this latter recursion scheme $\text{src}_{\mathcal{C}}$, e.g., if we need to distinguish it from $\text{snrc}_{\mathcal{C}}$.

Note that the version of safe composition along a normal parameter above differs from the previous one, Item from Definition 33, since the function h is allowed to use oracles. Again, this difference is inessential in terms of computational complexity, as we shall see. However, as we have mentioned, the greater expressivity of $B^{\mathcal{C}}$ and $NB^{\mathcal{C}}$ will facilitate our overall strategy for characterising CB and CNB, cf. Figure 1.

Let us take a moment to point out that $NB^{\mathcal{C}}(\vec{a}) \supseteq B^{\mathcal{C}}(\vec{a})$ indeed contain only well-defined total functions over the oracles $\vec{a}$, by reduction to induction on $\mathcal{C}$.

4.3 Simultaneous Recursion Schemes

For our main results, we will ultimately need a typical property of function algebras, that $B^{\mathcal{C}}$ and $NB^{\mathcal{C}}$ are closed under simultaneous versions of their recursion schemes.

Definition 39 (Simultaneous schemes). We define schemes $\text{ssrc}_{\mathcal{C}}$ and $\text{ssnrc}_{\mathcal{C}}$, respectively, as follows, for arbitrary $\vec{a} = a_1, \dots, a_k$:

– from $h_i(\vec{a})(\vec{x}; \vec{y})$ over $\vec{a}, \vec{b}$, for $1 \leq i \leq k$, define $f_i(\vec{x}; \vec{y})$ over $\vec{b}$, for $1 \leq i \leq k$, by:

$$f_i(\vec{x}; \vec{y}) = h_i((\lambda \vec{u} \subset \vec{x}, \lambda \vec{v} \subseteq \vec{y}. f_j(\vec{u}; \vec{v}))_{1 \leq j \leq k})(\vec{x}; \vec{y}).$$

– from $h_i(\vec{a})(\vec{x}; \vec{y})$ over $\vec{a}, \vec{b}$, for $1 \leq i \leq k$, define $f_i(\vec{x}; \vec{y})$ over $\vec{b}$, for $1 \leq i \leq k$, by:

$$f_i(\vec{x}; \vec{y}) = h_i((\lambda \vec{u} \subset \vec{x}, \lambda \vec{v} \subseteq \vec{y}. f_j(\vec{u}; \vec{v}))_{1 \leq j \leq k})(\vec{x}; \vec{y}).$$

PROPOSITION 40. *We have the following:*

- (1) If $\vec{f}(\vec{x}; \vec{y})$ over $\vec{b}$ are obtained by applying $\text{ssrc}_{\mathcal{C}}$ to $\vec{h}(\vec{a})(\vec{x}; \vec{y}) \in B^{\mathcal{C}}(\vec{a}, \vec{b})$, then also $\vec{f}(\vec{x}; \vec{y}) \in B^{\mathcal{C}}(\vec{b})$.
- (2) If $\vec{f}(\vec{x}; \vec{y})$ over $\vec{b}$ are obtained by applying $\text{ssnrc}_{\mathcal{C}}$ to $\vec{h}(\vec{a})(\vec{x}; \vec{y}) \in NB^{\mathcal{C}}(\vec{a}, \vec{b})$, then also $\vec{f}(\vec{x}; \vec{y}) \in NB^{\mathcal{C}}(\vec{b})$.

PROOF. We only prove Item 1, i.e., that $B^{\mathcal{C}}$ is closed under $\text{ssrc}_{\mathcal{C}}$, but the same argument works for Item 2: just ignore all guards on safe inputs in recursive calls.

Let $f_i(\vec{x}; \vec{y})$ and $h_i(a_1, \dots, a_k)(\vec{x}; \vec{y})$ be as given in Definition 39, and temporarily write $f_j^{\vec{x}; \vec{y}}$ for $\lambda \vec{u} \subset \vec{x}, \lambda \vec{v} \subseteq \vec{y}. f_j(\vec{u}; \vec{v})$, so we have:

$$f_i(\vec{x}; \vec{y}) = h_i(f_1^{\vec{x}; \vec{y}}, \dots, f_k^{\vec{x}; \vec{y}})(\vec{x}; \vec{y}).$$

For $i \in \mathbb{N}$, let us temporarily write $\underline{i}$ for i in binary notation,³ and $\vec{i}$ for the list $\underline{i}, \underline{i+1}, \dots, \underline{k}, \underline{1}, \underline{2}, \dots, \underline{i-1}$. Note that, for all $i = 1, \dots, k$, $\vec{i}$ is a permutation (in fact a rotation) of $\underline{1}, \dots, \underline{k}$.

Now, let $f(\vec{x}; \vec{y}, \vec{z})$ over oracles $\vec{b}$ be given as follows:

$$f(\vec{x}; \vec{y}, \vec{z}) := \begin{cases} h_1(f_1^{\vec{x}; \vec{y}}, \dots, f_k^{\vec{x}; \vec{y}})(\vec{x}; \vec{y}) & \vec{z} = \vec{1} \\ \vdots & \\ h_k(f_1^{\vec{x}; \vec{y}}, \dots, f_k^{\vec{x}; \vec{y}})(\vec{x}; \vec{y}) & \vec{z} = \vec{k} \\ 0 & \text{otherwise.} \end{cases} \quad (6)$$

³In fact, any notation will do, but we pick one for concreteness.

Note that this really is a finite case distinction since each of the boundedly many $\vec{i}$ has bounded size, both bounds depending only on k , and so is computable in B^- over $\vec{h}$.

By definition, then, we have that $f(\vec{x}; \vec{y}, \vec{i}) = f_i(\vec{x}; \vec{y})$. Moreover note that, for each $j = 1, \dots, k$, we have,

$$\begin{aligned} f_j^{\vec{x}; \vec{y}}(\vec{u}'; \vec{v}') &= (\lambda \vec{u} \subset \vec{x}, \lambda \vec{v} \subseteq \vec{y}. f_j(\vec{u}; \vec{v}))(\vec{u}'; \vec{v}') \\ &= (\lambda \vec{u} \subset \vec{x}, \lambda \vec{v} \subseteq \vec{y}. f(\vec{u}; \vec{v}, \vec{j}))(\vec{u}'; \vec{v}') \\ &= (\lambda \vec{u} \subset \vec{x}, \lambda \vec{v} \subseteq \vec{y}, \lambda \vec{w} \subseteq \vec{z}. f(\vec{u}; \vec{v}, \vec{w}))(\vec{u}'; \vec{v}', \vec{j}), \end{aligned}$$

as long as $\vec{z}$ is some $\vec{i}$, so indeed (6) has the form:

$$f(\vec{x}; \vec{y}, \vec{z}) = h(\lambda \vec{u} \subset \vec{x}, \lambda \vec{v} \subseteq \vec{y}, \lambda \vec{w} \subseteq \vec{z}. f(\vec{u}; \vec{v}, \vec{w}))(\vec{x}; \vec{y}),$$

and $f(\vec{x}; \vec{y}, \vec{z}) \in B^c(\vec{b})$ by srec_c . Finally, since $f_i(\vec{x}; \vec{y}) = f(\vec{x}; \vec{y}, \vec{i})$, we indeed have that each $f_i(\vec{x}; \vec{y}) \in B^c(\vec{b})$. $\square$

5 Characterisations for Function Algebras

In this section, we characterise the complexities of the function algebras we introduced in the previous section. Namely, despite apparently extending B , B^c still contains just the polynomial-time functions, whereas both NB and NB^c are shown to contain just the elementary functions. All such results rely on a ‘bounding lemma’ inspired by [6].

5.1 A Relativised Bounding Lemma

Bellantoni and Cook showed in [6] that any function $f(\vec{x}; \vec{y}) \in B$ satisfies the ‘poly-max bounding lemma’: there is a polynomial $p_f(\vec{n})$ such that:⁴

$$|f(\vec{x}; \vec{y})| \leq p_f(|\vec{x}|) + \max |\vec{y}|. \quad (7)$$

This provided a suitable invariant for eventually proving that all B -functions were polynomial-time computable.

In this work, inspired by that result, we generalise the bounding lemma to a form suitable to the relativised algebras from the previous section. To this end, we establish in the next result a sort of ‘elementary-max’ bounding lemma that accounts for the usual poly-max bounding as a special case, by appealing to the notion of (un)nested safe composition. Both the statement and the proof are quite delicate due to our algebras’ formulation using oracles; we must assume an appropriate bound for the oracles themselves, and the various (mutual) dependencies in the statement are subtle.

To state and prove the Bounding Lemma, let us employ the notation $\|\vec{x}\| := \sum |\vec{x}|$.

LEMMA 41 (BOUNDING LEMMA). *Let $f(\vec{a})(\vec{x}; \vec{y}) \in NB^c(\vec{a})$, with $\vec{a} = a_1, \dots, a_k$, and suppose $g_1, \dots, g_k$ are safe-normal functions. Then, there is an elementary function $e_f(n)$ and a constant $d_f \geq 1$ such that whenever there are constants $\vec{c} = c_1, \dots, c_k$ satisfying, for $1 \leq i \leq k$:*

$$|g_i(\vec{u}; \vec{v})| \leq c_i + \left(d_f \sum_{j \neq i} c_j \right) + \max |\vec{v}| \quad \text{for all inputs } \vec{u}, \vec{v}, \quad (8)$$

*then we have the following, for all inputs $\vec{u}, \vec{v}$:*⁵

$$|f(\vec{g})(\vec{u}; \vec{v})| \leq m_f(\vec{c}, \vec{u}, \vec{v}) \quad (9)$$

$$f(\vec{g})(\vec{u}; \vec{v}) = f(\lambda \vec{x}. \lambda |\vec{y}| \leq m_f(\vec{c}, \vec{u}, \vec{v}). g_i(\vec{x}; \vec{y}))_{1 \leq i \leq k}(\vec{u}; \vec{v}), \quad (10)$$

⁴Recall that, for $\vec{x} = x_1, \dots, x_n$, we write $|\vec{x}|$ for $|x_1|, \dots, |x_n|$.

⁵To be clear, we write $|\vec{y}| \leq m_f(\vec{c}, \vec{u}, \vec{v})$ here as an abbreviation for $\{|y_j| \leq m_f(\vec{c}, \vec{u}, \vec{v})\}_j$.

where

$$m_f(\vec{c}, \vec{x}, \vec{y}) = e_f(\|\vec{x}\|) + d_f \sum \vec{c} + \max |\vec{y}|.$$

Moreover, if in fact $f(\vec{x}; \vec{y}) \in B^c(\vec{a})$, then $d_f = 1$ and $e_f(n)$ is a polynomial.

Notice that in the above statement we employ the notation $\lambda \vec{x}, \lambda |\vec{y}| \leq n$, where $\vec{y} = y_1, \dots, y_k$, for the guarded abstraction defined by:

$$(\lambda \vec{x}, \lambda |\vec{y}| \leq n. h(\vec{x}; \vec{y}))(\vec{w}; \vec{z}) := \begin{cases} h(\vec{w}; \vec{z}) & \text{if } |z_j| \leq n \text{ for all } 1 \leq j \leq k \\ 0 & \text{otherwise.} \end{cases}$$

Unwinding the statement above, note that e_f and d_f depend *only* on the function f itself, not on the constants $\vec{c}$ given for the (mutual) oracle bounds in Equation (8). This is crucial for the proof, namely in the case when f is defined by recursion, substituting different values for $\vec{c}$ during an inductive argument.

While the role of the elementary bounding function e_f is a natural counterpart of p_f in Bellantoni and Cook's bounding lemma, cf. Equation (7), the role of d_f is perhaps slightly less clear. Intuitively, d_f represents the amount of 'nesting' in the definition of f , increasing whenever oracle calls are substituted into arguments for other oracles. Hence, if f uses only unnested Safe Composition, then $d_f = 1$ as required. In fact, it is only important to distinguish whether $d_f = 1$ or not, since d_f forms the base of an exponent for defining e_f when f is defined by safe recursion.

Finally let us note that Equations (9) and (10) are somewhat dual: while the former bounds the *output* of a function (serving as a modulus of *growth*), the latter bounds the *inputs* (serving as a modulus of *continuity*).

Remark 42. Let us also point out that we may relativise the statement of the lemma to any set of oracles including those which $f(\vec{x}; \vec{y})$ is over. In particular, if $f(\vec{x}; \vec{y})$ is over no oracles, then we may realise Equation (8) vacuously by choosing $\vec{a} = \emptyset$ and we would obtain that $|f(\vec{x}; \vec{y})| \leq e_f(\|\vec{x}\|) + \max |\vec{y}|$. More interestingly, in the case when $f(\vec{x}; \vec{y})$ is just, say, $a_i(\vec{x}; \vec{y})$, we may choose to set $\vec{a} = a_i$ or $\vec{a} = a_1, \dots, a_n$ in the above lemma, yielding different bounds in each case. We shall exploit this in inductive hypotheses in the proof that follows (typically when we write 'WLoG').

PROOF OF LEMMA 41. We prove Equations (9) and (10) by induction on the definition of $f(\vec{x}; \vec{y})$, always assuming that we have $\vec{c}$ satisfying Equation (8).

Throughout the argument we shall actually construct $e_f(n)$ that are *monotone* elementary functions (without loss of generality) and exploit this invariant. In fact $e_f(n)$ will always be generated by composition from 0, s, +, $\times$, x^y and projections. If $f(\vec{x}; \vec{y}) \in B^c(\vec{a})$ then $e_f(n)$ will be in the same algebra without exponentiation, x^y , i.e., it will be a polynomial with only non-negative coefficients. This property will be made clear by the given explicit definitions of $e_f(n)$ throughout the argument.

Let us start with Equation (9). If $f(\vec{x}; \vec{y})$ is an initial function then it suffices to set $e_f(n) := 1 + n$ and $d_f := 1$.

If $f(\vec{x}; \vec{y}) = a_i(\vec{x}; \vec{y})$ then it suffices to set $e_f(n) := 0$ and $d_f := 1$.

If $f(\vec{x}; \vec{y}) = h(\vec{x}; \vec{y}, g(\vec{x}; \vec{y}))$, let e_h, e_g, d_h, d_g be obtained from the inductive hypothesis. We have:

$$\begin{aligned} |f(\vec{x}; \vec{y})| &= |h(\vec{x}; \vec{y}, g(\vec{x}; \vec{y}))| \\ &\leq e_h(\|\vec{x}\|) + d_h \sum \vec{c} + \max(|\vec{y}|, |g(\vec{x}; \vec{y})|) \\ &\leq e_h(\|\vec{x}\|) + d_h \sum \vec{c} + e_g(\|\vec{x}\|) + d_g \sum \vec{c} + \max |\vec{y}|, \end{aligned}$$

so we may set $e_f(n) := e_h(n) + e_g(n)$ and $d_f := d_h + d_g$.

For the ‘moreover’ clause, note that if $f(\vec{x}; \vec{y})$ uses only unnnested Safe Composition, then one of g or h does not have oracles, and so we can assume WLoG that either the term $d_h \sum \vec{c}$ or the term $d_g \sum \vec{c}$ above does not occur, and we set $d_f := d_g$ or $d_f := d_h$, respectively. In either case we obtain $d_f = 1$ by the inductive hypothesis, as required.

If $f(\vec{x}; \vec{y}) = h(\vec{x}, g(\vec{x}; \cdot); \vec{y})$, let e_h, e_g, d_h, d_g be obtained from the inductive hypothesis. Note that, by definition of Safe Composition along a normal parameter, we must have that g has no oracles, and so in fact $|g(\vec{x}; \cdot)| \leq e_g(\|\vec{x}\|)$. We thus have,

$$\begin{aligned} |f(\vec{x}; \vec{y})| &= |h(\vec{x}, g(\vec{x}; \cdot); \vec{y})| \\ &\leq e_h(\|\vec{x}\|, |g(\vec{x}; \cdot)|) + d_h \sum \vec{c} + \max |\vec{y}| \\ &\leq e_h(\|\vec{x}\|, e_g(\|\vec{x}\|)) + d_h \sum \vec{c} + \max |\vec{y}|, \end{aligned}$$

so we may set $e_f(n) := e_h(n, e_g(n))$ and $d_f := d_h$. For the ‘moreover’ clause, note that by the inductive hypothesis e_h, e_g are polynomials and $d_h = 1$, so indeed e_f is a polynomial and $d_f = 1$.

Finally, if $f(\vec{x}; \vec{y}) = h(\lambda \vec{u} \subset \vec{x}, \lambda \vec{v}. f(\vec{u}; \vec{v}))(\vec{x}; \vec{y})$, let e_h, d_h be obtained from the inductive hypothesis. We claim that it suffices to set $d_f := d_h$ and $e_f(n) := nd_h^n e_h(n)$. Note that, for the ‘moreover’ clause, if $d_h = 1$ then also $d_h^n = 1$ and so indeed $e_f(n)$ is a polynomial if $d_h = 1$ and $e_h(n)$ is a polynomial.

First, let us calculate the following invariant, for $n > 0$:

$$\begin{aligned} e_f(n) &= nd_h^n e_h(n) \\ &= d_h^n e_h(n) + (n-1)d_h^n e_h(n) \\ &= d_h^n e_h(n) + d_h(n-1)d_h^{n-1} e_h(n) \\ &\geq e_h(n) + d_h(n-1)d_h^{n-1} e_h(n) && d_h \geq 1 \\ &\geq e_h(n) + d_h(n-1)d_h^{n-1} e_h(n-1) && e_h \text{ monotone} \\ &\geq e_h(n) + d_h e_f(n-1) && \text{def. of } e_f \end{aligned}$$

Hence:

$$e_f(n) \geq e_h(n) + d_h e_f(n-1). \quad (11)$$

Now, to show that Equation (9) bounds the f, e_f, d_f at hand, we proceed by a sub-induction on $\|\vec{x}\|$. For the base case, when $\|\vec{x}\| = 0$ (and so, indeed, $\vec{x} = \vec{0}$), note simply that $\lambda \vec{u} \subset \vec{x}, \lambda \vec{v}. f(\vec{u}; \vec{v})$ is the constant function 0, and so we may appeal to the main inductive hypothesis for $h(a)$ setting the corresponding constant c for a to be 0 to obtain:

$$\begin{aligned} |f(\vec{0}; \vec{y})| &= |h(0)(\vec{0}; \vec{y})| \\ &\leq e_h(0) + d_h \sum \vec{c} + \max |\vec{y}| \\ &\leq e_f(0) + d_f \sum \vec{c} + \max |\vec{y}|, \end{aligned}$$

as required. For the sub-inductive step, let $\|\vec{x}\| > 0$. Note that, whenever $\vec{u} \subset \vec{x}$ we have $\|\vec{u}\| < \|\vec{x}\|$ and so, by the sub-inductive hypothesis and monotonicity of e_f we have:

$$|f(\vec{u}; \vec{v})| \leq e_f(\|\vec{x}\| - 1) + d_f \sum \vec{c} + \max |\vec{v}|.$$

Now, we may again appeal to the main inductive hypothesis for $h(a)$ by setting $c = e_f(\|\vec{x}\| - 1)$ to be the corresponding constant for $a = \lambda\vec{u} \subset \vec{x}, \lambda\vec{v}.f(\vec{u}; \vec{v})$. We thus obtain:

$$\begin{aligned}
 |f(\vec{x}; \vec{y})| &= |h(\lambda\vec{u} \subset \vec{x}, \lambda\vec{v}.f(\vec{u}; \vec{v}))(\vec{x}; \vec{y})| \\
 &\leq e_h(\|\vec{x}\|) + d_h c + d_h \sum \vec{c} + \max |\vec{y}| && \text{main IH} \\
 &\leq (e_h(\|\vec{x}\|) + d_h e_f(\|\vec{x}\| - 1)) + \\
 &\quad + d_h \sum \vec{c} + \max |\vec{y}| && \text{def. of } c \\
 &\leq e_f(\|\vec{x}\|) + d_h \sum \vec{c} + \max |\vec{y}| && (11) \\
 &\leq e_f(\|\vec{x}\|) + d_f \sum \vec{c} + \max |\vec{y}| && d_f = d_h.
 \end{aligned}$$

Let us now prove Equation (10) and let e_f and d_f be constructed by induction on f as above. We proceed again by induction on the definition of $f(\vec{a})(\vec{x}; \vec{y})$, always making explicit the oracles of a function.

The initial functions and oracle calls are immediate, due to the ‘ $\max |\vec{y}|$ ’ term in Equation (10).

If $f(\vec{a})(\vec{x}; \vec{y}) = h(\vec{a})(\vec{x}; \vec{y}, g(\vec{a})(\vec{x}; \vec{y}))$ then, by the inductive hypothesis for $h(\vec{a})$, any oracle call from $h(\vec{a})$ only takes safe inputs of lengths:

$$\begin{aligned}
 &\leq e_h(\|\vec{x}\|) + d_h \sum \vec{c} + \max(|\vec{y}|, |g(\vec{a})(\vec{x}; \vec{y})|) \\
 &\leq e_h(\|\vec{x}\|) + d_h \sum \vec{c} + e_g(\|\vec{x}\|) + d_g \sum \vec{c} + \max |\vec{y}| && (8) \\
 &\leq (e_h(\|\vec{x}\|) + e_g(\|\vec{x}\|)) + (d_h + d_g) \sum \vec{c} + \max |\vec{y}| \\
 &\leq e_f(\|\vec{x}\|) + d_f \sum \vec{c} + \max |\vec{y}|.
 \end{aligned}$$

Note that any oracle call from $g(\vec{a})$ will still only take safe inputs of lengths $\leq e_g(\|\vec{x}\|) + d_g \sum \vec{c} + \max |\vec{y}|$, by the inductive hypothesis, and e_g and d_g are bounded above by e_f and d_f , respectively.

If $f(\vec{a})(\vec{x}; \vec{y}) = h(\vec{a})(\vec{x}, g(\emptyset)(\vec{x}; \vec{y}))$ then, by the inductive hypothesis, any oracle call will only take safe inputs of lengths:

$$\begin{aligned}
 &\leq e_h(\|\vec{x}\| + |g(\emptyset)(\vec{x}; \vec{y})|) + d_h \sum \vec{c} + \max |\vec{y}| \\
 &\leq e_h(\|\vec{x}\| + e_g(\|\vec{x}\|)) + d_h \sum \vec{c} + \max |\vec{y}| && \text{Lemma 41} \\
 &\leq e_f(\|\vec{x}\|) + d_f \sum \vec{c} + \max |\vec{y}|.
 \end{aligned}$$

Last, suppose $f(\vec{a})(\vec{x}; \vec{y}) = h(\vec{a}, \lambda\vec{u} \subset \vec{x}, \lambda\vec{v}.f(\vec{a})(\vec{u}; \vec{v}))(\vec{x}; \vec{y})$. We proceed by a sub-induction on $\|\vec{x}\|$. Note that, since $\vec{u} \subset \vec{x} \implies \|\vec{u}\| < \|\vec{x}\|$, we immediately inherit from the inductive hypothesis the appropriate bound on safe inputs for oracle calls from $\lambda\vec{u} \subset \vec{x}, \lambda\vec{v}.f(\vec{a})(\vec{u}; \vec{v})$.

Now, recall from the Bounding Lemma 41, whenever $\vec{u} \subset \vec{x}$ (and so $\|\vec{u}\| < \|\vec{x}\|$), we have $|f(\vec{u}; \vec{v})| \leq e_f(\|\vec{x}\| - 1) + d_f \sum \vec{c} + \max |\vec{v}|$. So by setting $c = e_f(\|\vec{x}\| - 1)$ in the inductive hypothesis for $h(\vec{a}, a)$, with $a = \lambda\vec{u} \subset \vec{x}, \lambda\vec{v}.f(\vec{a})(\vec{u}; \vec{v})$, any oracle call from $h(\vec{a}, a)$ will only take safe inputs of lengths:

$$\begin{aligned}
 &\leq e_h(\|\vec{x}\|) + d_h e_f(\|\vec{x}\| - 1) + d_h \sum \vec{c} + \max |\vec{y}| \\
 &\leq e_f(\|\vec{x}\|) + d_f \sum \vec{c} + \max |\vec{y}| && (11).
 \end{aligned}$$

This completes the proof. $\square$

5.2 Soundness Results

In this subsection, we show that the function algebras B^c and NB^c (as well as NB) capture precisely the classes **FPTIME** and **FELEMENTARY**, respectively. We start with B^c :

THEOREM 43. *Suppose $\vec{a}$ satisfies Equation (8) for some constants $\vec{c}$. We have the following:*

- (1) *If $f(\vec{x}; \vec{y}) \in B^c(\vec{a})$ then $f(\vec{x}; \vec{y}) \in \mathbf{FPTIME}(\vec{a})$.*
- (2) *If $f(\vec{x}; \vec{y}) \in NB^c(\vec{a})$ then $f(\vec{x}; \vec{y}) \in \mathbf{FELEMENTARY}(\vec{a})$.*

Note in particular that, for $f(\vec{x}; \vec{y})$ in B^C or NB^C , i.e., not using any oracles, we immediately obtain membership in **FPTIME** or **FELEMENTARY**, respectively. However, the reliance on intermediate oracles during a function definition causes some difficulties that we must take into account. At a high level, the idea is to use the Bounding Lemma (namely Equation (10)) to replace certain oracle calls with explicit appropriately bounded functions computing their graphs. From here, we compute $f(\vec{x}; \vec{y})$ by a sort of ‘course-of-values’ recursion on $\subset$, storing previous values in a lookup table. In the case of B^C , it is important that this table has polynomial-size, since there are only $m! \prod |\vec{x}|$ permutations of prefixes of a list $\vec{x} = x_1, \dots, x_m$ (which is a polynomial of degree m).

PROOF OF THEOREM 43. We proceed by induction on the definition of $f(\vec{x}; \vec{y})$.

Each initial function is polynomial-time computable, and each (relativised) complexity class considered is under composition, so it suffices to only consider the respective recursion schemes. We shall focus first on the case of $B^C(\vec{a})$, Item 1 above, so that e_f is a polynomial and $d_f = 1$.

Suppose we have $h(a)(\vec{x}; \vec{y}) \in B^C(a, \vec{a})$ and let:

$$f(\vec{x}; \vec{y}) = h(\lambda \vec{u} \subset \vec{x}, \lambda \vec{v} \subseteq \vec{y}. f(\vec{u}; \vec{v}))(\vec{x}; \vec{y}).$$

We start by making some observations:

- (1) First, note that $|f(\vec{x}; \vec{y})| \leq e_f(|\vec{x}|) + d_f \sum \vec{c} + \max |\vec{y}|$, by the Bounding Lemma 41, and so $|f(\vec{x}; \vec{y})|$ is polynomial in $|\vec{x}|, |\vec{y}|$.
- (2) Second, note that the set $[\vec{x}; \vec{y}] := \{(\vec{u}, \vec{v}) \mid \vec{u} \subset \vec{x}, \vec{v} \subseteq \vec{y}\}$ has size polynomial in $|\vec{x}|, |\vec{y}|$:
 - write $\vec{x} = x_1, \dots, x_m$ and $\vec{y} = y_1, \dots, y_n$.
 - Each x_i and y_j have only linearly many prefixes, and so there are at most $|x_1| \cdots |x_m| |y_1| \cdots |y_n| \leq \|\vec{x}, \vec{y}\|^{m+n}$ many choices of prefixes for all the arguments $\vec{x}, \vec{y}$ (This is a polynomial since m and n are global constants).
 - Additionally, there are $m!$ permutations of the arguments $\vec{x}$ and $n!$ permutations of the arguments $\vec{y}$. Again, since m and n are global constants, we indeed have $|[\vec{x}; \vec{y}]| = O(\|\vec{x}, \vec{y}\|^{m+n})$, which is polynomial in $|\vec{x}|, |\vec{y}|$.

We describe a polynomial-time algorithm for computing $f(\vec{x}; \vec{y})$ (over oracles $\vec{a}$) by a sort of ‘course-of-values’ recursion on the order $\subset \times \subseteq$ on $[\vec{x}; \vec{y}]$.

First, for convenience, temporarily extend $\subset \times \subseteq$ to a total well-order on $[\vec{x}; \vec{y}]$, and write S for the associated successor function. Note that S can be computed in polynomial-time from $[\vec{x}; \vec{y}]$.

Define $F(\vec{x}, \vec{y}) := \langle f(\vec{u}; \vec{v}) \rangle_{\vec{u} \subset \vec{x}, \vec{v} \subseteq \vec{y}}$, i.e., it is the graph of $\lambda \vec{u} \subset \vec{x}, \lambda \vec{v} \subseteq \vec{y}. f(\vec{u}; \vec{v})$ that we shall use as a ‘lookup table’. Note that $|F(\vec{x}, \vec{y})|$ is polynomial in $|\vec{x}|, |\vec{y}|$ by Item 1 and Item 2 above. Now, we can write⁶:

$$\begin{aligned} F(S(\vec{x}, \vec{y})) &= \langle f(S(\vec{x}, \vec{y})), F(\vec{x}, \vec{y}) \rangle \\ &= \langle h(F(\vec{x}, \vec{y}))(\vec{x}; \vec{y}), F(\vec{x}, \vec{y}) \rangle. \end{aligned}$$

Again by Item 2 (and since F is polynomially bounded), this recursion terminates in polynomial-time. We may now simply calculate $f(\vec{x}; \vec{y})$ as $h(F(\vec{x}, \vec{y}))(\vec{x}; \vec{y})$.

The argument for NB^C is similar, though we need not be as careful about computing the size of the lookup tables (F above) for recursive calls. The key idea is to use the Bounding Lemma (Equation 9) to bound the safe inputs of recursive calls so that we can adequately store the lookup table for previous values. $\square$

⁶Here, as abuse of notation, we are now simply identifying $F(\vec{x}; \vec{y})$ with $\lambda \vec{u} \subset \vec{x}, \lambda \vec{v} \subseteq \vec{y}. f(\vec{x}; \vec{y})$.

5.3 Completeness and Characterisations

We are now ready to give our main function algebraic characterisation results for polynomial-time:

COROLLARY 44. *The following are equivalent:*

- (1) $f(\vec{x};) \in \mathbb{B}$.
- (2) $f(\vec{x};) \in \mathbb{B}^\subset$.
- (3) $f(\vec{x}) \in \mathbf{FPTIME}$.

PROOF. (1) $\implies$ (2) is trivial, and (2) $\implies$ (3) is given by Theorem 43. (1). Finally, (3) $\implies$ (1) is from [6], stated in Theorem 2 earlier. $\square$

The remainder of this subsection is devoted to establishing a similar characterisation for NB, $\mathbf{NB}^\subset$ and **FELEMENTARY**.

To begin with, we recall the definition of the class **FELEMENTARY**:

Definition 45. **FELEMENTARY** is the smallest set of functions containing:

- $0() := 0 \in \mathbb{N}$,
- $\pi_i^n(x_1, \dots, x_n) := x_j$, whenever $1 \leq j \leq n$;
- $s(x) := x + 1$;
- the function E_2 defined as follows:

$$E_1(x) = x^2 + 2$$

$$E_2(0) = 2$$

$$E_2(x + 1) = E_1(E_2(x)),$$

and closed under the following:

- (Composition) If $f(\vec{x}, x), g(\vec{x}) \in \mathbf{FELEMENTARY}$ then so is $f(\vec{x}, g(\vec{x}))$;
- (Bounded recursion) If $g(\vec{x}), h(x, \vec{x}, y), j(x, \vec{x})$ are functions in **FELEMENTARY** then so is $f(x, \vec{x})$ given by:

$$f(0, \vec{x}) := g(\vec{x})$$

$$f(x + 1, \vec{x}) := h(x, \vec{x}, f(x, \vec{x})),$$

provided that $f(x, \vec{x}) \leq j(x, \vec{x})$.

PROPOSITION 46 ([33]). *Let $f \in \mathbf{FELEMENTARY}$ be a k -ary function. Then, there exists an integer m such that:*

$$f(\vec{x}) \leq E_2^m(\max_k(\vec{x})),$$

where $E_2^0(x) = x$ and $E_2^{m+1}(x) = E_2(E_2^m(x))$.

For our purposes we shall consider a formulation of this class in binary notation, that we call **FELEMENTARY**_{0,1}.

Definition 47. **FELEMENTARY**_{0,1} is the smallest set of functions containing:

- $0() := 0 \in \mathbb{N}$,
- $\pi_i^n(x_1, \dots, x_n) := x_j$, whenever $1 \leq j \leq n$;
- $s_i(x) := 2x + i$, for $i \in \{0, 1\}$

– the function $\varepsilon(x, y)$ defined as follows:

$$\begin{aligned}\varepsilon(0, y) &:= s_0(y) \\ \varepsilon(s_i x, y) &:= \varepsilon(x, \varepsilon(x, y))\end{aligned}$$

and closed under the following:

- (Composition) If $f(\vec{x}, x), g(\vec{x}) \in \mathbf{FELEMENTARY}_{0,1}$ then so is $f(\vec{x}, g(\vec{x}))$;
- (Bounded recursion on notation) If $g(\vec{x}), h_i(x, \vec{x}, y), j(x, \vec{x})$ are functions in $\mathbf{FELEMENTARY}_{0,1}$ then so is $f(x, \vec{x})$ given by:

$$\begin{aligned}f(0, \vec{x}) &:= g(\vec{x}) \\ f(s_i x, \vec{x}) &:= h_i(x, \vec{x}, f(x, \vec{x})),\end{aligned}$$

provided that $f(x, \vec{x}) \leq j(x, \vec{x})$.

It is easy to show that the unary and the binary definition of the class of elementary time computable functions coincide. To see this, we first define $\varepsilon^n(x)$ as

$$\begin{aligned}\varepsilon^1(x) &:= \varepsilon(x, 1) \\ \varepsilon^{m+1}(x) &:= \varepsilon^1(\varepsilon^m(x)),\end{aligned}\tag{12}$$

which allows us to prove that $\varepsilon^n(x)$ plays the role of rate growth function as the function $E_2^n(x)$ (see Proposition 46).

PROPOSITION 48.

- (1) $\mathbf{FELEMENTARY} = \mathbf{FELEMENTARY}_{0,1}$;
- (2) for any $f \in \mathbf{FELEMENTARY}_{0,1}$ k -ary function there is an integer m such that:

$$f(\vec{x}) \leq \varepsilon^m(\max_k(\vec{x})).$$

PROOF. Let us first prove point 1. For the $\supseteq$ direction we show that for any $f \in \mathbf{FELEMENTARY}_{0,1}$ there exists n such that:

$$|f(\vec{x})| \leq 2_n \left(\sum |\vec{x}| \right),\tag{13}$$

where $2_0(x) = x$ and $2_{n+1}(x) = 2^{2_n(x)}$. Since $|x| = \lceil \log_2(x+1) \rceil$, from the above inequation we would have that, for some m :

$$f(\vec{x}) \leq 2_{n+m} \left(\sum \vec{x} \right),$$

which allows us to conclude $f \in \mathbf{FELEMENTARY}$, as the elementary time computable functions are exactly the elementary space ones. The inequation (13) can be proved by induction on f , noticing that $|\varepsilon(x, y)| = 2^{|x|} + |y|$. Concerning the $\subseteq$ direction, we prove by induction on $f \in \mathbf{FELEMENTARY}$ that there exists a function $\hat{f} \in \mathbf{FELEMENTARY}_{0,1}$ such that, for all $\vec{x} = x_1, \dots, x_n$:

$$f(\vec{x}) = |\hat{f}(x_1, \dots, x_n)|,$$

where $\underline{m} := s_1^m(0) = s_1(\dots s_1(0))$. Since the functions $|\cdot|$ and $x \mapsto \underline{x}$ are both in $\mathbf{FELEMENTARY}_{0,1}$, we are able to conclude $f \in \mathbf{FELEMENTARY}_{0,1}$. The case $f = 0$ is trivial. As for the cases $f = s$ and $f = \pi_i^n$, we first notice that $|\underline{x}| = x$. Then, we have:

$$\begin{aligned}s(x) &= |s(x)| \\ \pi_i^n(\vec{x}) &= x_i = |\underline{x}_i| = |\pi_i^n(\underline{x}_1, \dots, \underline{x}_n)|.\end{aligned}$$

Concerning the case of $E_2(x)$, we first notice that the following property holds for any m and some k :

$$E_2^m(x) \leq \varepsilon^{m+k}(x), \quad (14)$$

where $\varepsilon^n(x)$ is as in (12). Moreover, the function $E_2(x)$ can be defined by two applications of bounded recursion proceeding from the successor and the projection functions, where each recursion can be bounded by $E_2(x)$, and hence by $\varepsilon^k(x)$ for some k . This means that the case of $E_2(x)$ can be reduced to the case of bounded recursion. Suppose now that $f(\vec{x}) = h(\vec{x}, g(\vec{x}))$. We define $\hat{f}(\vec{x}) = \hat{h}(\vec{x}, \hat{g}(\vec{x}))$ so that, by induction hypothesis:

$$\begin{aligned} f(\vec{x}) &= h(\vec{x}, g(\vec{x})) \\ &= |\hat{h}(\underline{x}_1, \dots, \underline{x}_n, \underline{g(\vec{x})})| \\ &= |\hat{h}(\underline{x}_1, \dots, \underline{x}_n, |\hat{g}(\underline{x}_1, \dots, \underline{x}_n)|)| \\ &= |\hat{h}(\underline{x}_1, \dots, \underline{x}_n, \hat{g}(\underline{x}_1, \dots, \underline{x}_n))| \\ &= |\hat{f}(\vec{x})| = |\hat{h}(\vec{x}, \hat{g}(\vec{x}))|. \end{aligned}$$

Last, suppose that f has been obtained by bounded recursion from h, g, j , i.e.:

$$\begin{aligned} f(0, \vec{x}) &= g(\vec{x}) \\ f(y+1, \vec{x}) &= h(y, \vec{x}, f(y, \vec{x})), \end{aligned}$$

provided that $f(y, \vec{x}) \leq j(y, \vec{x})$. We define $\hat{f}$ as follows:

$$\begin{aligned} \hat{f}(0, \vec{x}) &= \hat{g}(\vec{x}) \\ \hat{f}(s_i y, \vec{x}) &= \hat{h}(y, \vec{x}, \hat{f}(y, \vec{x})). \end{aligned}$$

We show by induction on y that:

$$f(y, \vec{x}) = |\hat{f}(y, \underline{x}_1, \dots, \underline{x}_n)|.$$

We have:

$$\begin{aligned} f(0, \vec{x}) &= g(\vec{x}) \\ &= |\hat{g}(\underline{x}_1, \dots, \underline{x}_n)| \\ &= |\hat{f}(0, \underline{x}_1, \dots, \underline{x}_n)| \end{aligned}$$

$$\begin{aligned} f(y+1, \vec{x}) &= h(y, \vec{x}, f(y, \vec{x})) \\ &= |\hat{h}(\underline{y}, \underline{x}_1, \dots, \underline{x}_n, \underline{f(y, \vec{x})})| \\ &= |\hat{h}(\underline{y}, \underline{x}_1, \dots, \underline{x}_n, |\hat{f}(\underline{y}, \underline{x}_1, \dots, \underline{x}_n)|)| \\ &= |\hat{h}(\underline{y}, \underline{x}_1, \dots, \underline{x}_n, \hat{f}(\underline{y}, \underline{x}_1, \dots, \underline{x}_n))| \\ &= |\hat{f}(\underline{y+1}, \underline{x}_1, \dots, \underline{x}_n)|, \end{aligned}$$

since $f(y, \vec{x}) \leq j(y, \vec{x})$ and $j(y, \vec{x}) = |\hat{j}(\underline{y}, \underline{x}_1, \dots, \underline{x}_n)|$ by induction hypothesis, we are done.

Point 2 follows by point 1, Proposition 46 and (14). $\square$

Completeness for NB is based on a standard technique (see [6]), adapted to the case of **FELEMENTARY** in [35].

LEMMA 49. For any $f(\vec{x}) \in \mathbf{FELEMENTARY}_{0,1}$ there are a function $f^*(x; \vec{x}) \in \mathbf{NB}$ and a monotone function $t_f \in \mathbf{FELEMENTARY}_{0,1}$ such that for all integers $\vec{x}$ and all $w \geq t_f(\vec{x})$ we have $f^*(w; \vec{x}) = f(\vec{x})$.

PROOF. The proof is by induction on the definition of f . If f is the zero, successor or projection function then $f^* \in \mathbf{NB}$. In this case, we choose $t_f = 0$. The function ε has a definition by one application of bounded recursion on notation, proceeding from the successors and the projection functions, where each recursion is bounded by ε . Since the treatment of bounded recursion does not make use of the induction hypothesis for the bounding function, we can use this method to get functions $\varepsilon^* \in \mathbf{NB}$ and $t_\varepsilon \in \mathbf{FELEMENTARY}_{0,1}$ with the required properties. If $f(\vec{x}) = h(\vec{x}, g(\vec{x}))$ then we set $f^*(w; \vec{x}) = h^*(w; \vec{x}, g^*(w; \vec{x}))$, which is in $\mathbf{NB}$. Since the function g^* is clearly bounded by a monotone function $b \in \mathbf{FELEMENTARY}_{0,1}$, we set $t_f(\vec{x}) = t_h(\vec{x}, b(\vec{x})) + t_g(\vec{x})$, which is monotone. By applying the induction hypothesis, if $w \geq t_f(\vec{x})$ then:

$$f^*(w; \vec{x}) = h^*(w; \vec{x}, g^*(w; \vec{x})) = h^*(w; \vec{x}, g(\vec{x})) = h(\vec{x}, g(\vec{x})).$$

Let us finally suppose that $f(x, \vec{y})$ is defined by bounded recursion on notation from $g(\vec{y})$, $h_i(x, \vec{y})$, $f(x, \vec{y})$ and $j(x, \vec{y})$. By applying the induction hypothesis we set:

$$\begin{aligned} \hat{f}(0, w; x, \vec{y}) &= g^*(w; \vec{y}) \\ \hat{f}(s_i(x), w; x, \vec{y}) &= \text{cond}(W(s_i(v), w; x), g^*(w; \vec{y}), h_i^*(w; W(v, w; x), \vec{y}, f^*(v, w; x, \vec{y}))) \\ f^*(w; x, \vec{y}) &= \hat{f}(w, w; x, \vec{y}), \end{aligned}$$

where $W(v, w; x) = \dot{-}(\dot{-}(v; w); x)$ and $\dot{-}(x; y)$ is the truncated subtraction, which is in $\mathbf{B}$, and hence in $\mathbf{NB}$. We can easily show that $f^*(w; x, \vec{y}) \in \mathbf{NB}$. We define $t_f(x, \vec{y}) = t_g(\vec{y}) + \sum_i t_{h_i}(x, \vec{y}, j(x, \vec{y}))$, where j is the bounding function. Assuming j to be monotone, t_f is monotone too. We now show by induction on u that, whenever $w \geq t_f(x, \vec{y})$ and $w - x \leq u \leq w$:

$$\hat{f}(u, w; x, \vec{y}) = f(x - (w - u), \vec{y}). \quad (15)$$

If $u = w - x$ then we have two cases:

- if $u = 0$ then $\hat{f}(0, w; x, \vec{y}) = g^*(w; \vec{y})$;
- if $u = s_i(v)$ then, since $W(s_i(v), w; x) = 0$, we have $\hat{f}(s_i(v), w; x, \vec{y}) = g^*(w; \vec{y})$.

Hence, in any case:

$$\hat{f}(u, w; x, \vec{y}) = g^*(w; \vec{y}) = g(\vec{y}) = f(0; \vec{y}) = f(x - (w - u), \vec{y}).$$

Let us now suppose that $w - x < u \leq w$. This means that $u = s_i(v)$ and $W(s_i(v), w; x) > 0$. Moreover, by monotonicity of t_f and definition of j :

$$\begin{aligned} w &\geq t_f(x, \vec{y}) \\ &\geq t_f(x - (w - s_i(v)), \vec{y}) \\ &\geq t_f(x - (w - v), \vec{y}) \\ &\geq t_{h_i}(x - (w - v), \vec{y}, j(x - (w - v), \vec{y})) \\ &\geq t_{h_i}(x - (w - v), \vec{y}, f(x - (w - v), \vec{y})). \end{aligned}$$

By applying the induction hypothesis:

$$\begin{aligned} \hat{f}(s_i(v), w; x, \vec{y}) &= h_i^*(w; W(v, w; x), \vec{y}, \hat{f}(v, w; x, \vec{y})) \\ &= h_i^*(w; W(v, w; x), \vec{y}, f(x - (w - v), \vec{y})) \\ &= h_i^*(w; x - (w - v), \vec{y}, f(x - (w - v), \vec{y})) \end{aligned}$$

$$\begin{aligned}
&= h_i(x - (w - v), \vec{y}, f(x - (w - v), \vec{y})) \\
&= f(s_i((x - (w - v))), \vec{y}) \\
&= f((x - (w - s_i(v))), \vec{y}).
\end{aligned}$$

Now, by (15), for all $w \geq t_f(x, \vec{y})$ we have:

$$f^*(w; x, \vec{y}) = \hat{f}(w, w; x, \vec{y}) = f(x, \vec{y})$$

and this concludes the proof. $\square$

THEOREM 50. *If $f(\vec{x}) \in \mathbf{FELEMENTARY}$ then $f(\vec{x};) \in \mathbf{NB}$.*

PROOF. First, given the function $\text{ex}(x; y)$ in (4), we construct the function $\text{ex}^m(x;)$ by induction on m :

$$\begin{aligned}
\text{ex}^1(x;) &= \text{ex}(x; 1) \\
\text{ex}^{m+1}(x;) &= \text{ex}^m(\text{ex}^1(x;);).
\end{aligned}$$

Hence $\text{ex}^m(x) = \text{ex}^m(x;)$, for all $m \geq 1$. Now, let $f(\vec{x}) \in \mathbf{FELEMENTARY}$. By PROPOSITION 48.1, we have that $f(\vec{x}) \in \mathbf{FELEMENTARY}_{0,1}$. By Lemma 49, there exist $f^*(w; \vec{x}) \in \mathbf{NB}$ and a monotone function $t_f \in \mathbf{FELEMENTARY}_{0,1}$ such that, for all $w, \vec{x}$ with $w \geq t_f(\vec{x})$, it holds that $f^*(w; \vec{x}) = f(\vec{x})$. By Proposition 48.2 there exists $m \geq 1$ such that:

$$t_f(\vec{x}) \leq \text{ex}^m(\max_{\# \vec{x}}(\vec{x};);),$$

where $\max_{\# \vec{x}}(\vec{x};)$ is the k -ary maximum function, which is in $\mathbf{B}$ by Theorem 2, and hence in $\mathbf{NB}$. Therefore:

$$f(\vec{x};) = f^*(\text{ex}^m(\max_{\# \vec{x}}(\vec{x};);); \vec{x}) \in \mathbf{NB}.$$

$\square$

Now, by the same argument as for Corollary 44, only using Lemma 49 above instead of appealing to [6], we can give our main characterisation result for algebras for elementary computation:

COROLLARY 51. *The following are equivalent:*

- (1) $f(\vec{x};) \in \mathbf{NB}$.
- (2) $f(\vec{x};) \in \mathbf{NB}^\subset$.
- (3) $f(\vec{x}) \in \mathbf{FELEMENTARY}$.

6 Characterisations for Circular Systems

We now return our attention to the circular systems $\mathbf{CB}$ and $\mathbf{CNB}$ that we introduced in Section 3. We will address the complexity of their definable functions by ‘sandwiching’ them between function algebras of Section 4, given their characterisations that we have just established.

6.1 Completeness

To show that $\mathbf{CB}$ contains all polynomial-time functions, we may simply simulate Bellantoni and Cook’s algebra:

THEOREM 52. *If $f(\vec{x}; \vec{y}) \in \mathbf{B}$ then $f(\vec{x}; \vec{y}) \in \mathbf{CB}$.*

PROOF. By Proposition 6 it suffices to show that for any B-derivation $\mathcal{D}$ there is a CB-coderivation $\mathcal{D}^*$ such that $f_{\mathcal{D}}(\vec{x}; \vec{y}) = f_{\mathcal{D}^*}(\vec{x}, \vec{y})$. The proof is by induction on $\mathcal{D}$. The only non-trivial case is when $\mathcal{D}$ is the following derivation:

$$\frac{\begin{array}{ccc} \mathcal{D}_0 & \mathcal{D}_1 & \mathcal{D}_2 \\ \Gamma \Rightarrow N & \Box N, \Gamma, N \Rightarrow N & \Box N, \Gamma, N \Rightarrow N \end{array}}{\text{srec} \quad \Box N, \Gamma \Rightarrow N}$$

We define $\mathcal{D}^*$ as follows:

$$\frac{\begin{array}{c} \mathcal{D}_0^* \\ \Gamma \Rightarrow N \end{array} \quad \frac{\begin{array}{c} \vdots \\ \text{cond}_{\Box} \quad \Box N, \Gamma \Rightarrow \textcolor{red}{N} \end{array} \quad \frac{\begin{array}{c} \mathcal{D}_i^* \\ \Box N, \Gamma, \textcolor{red}{N} \Rightarrow N \end{array}}{\text{cut}_N \quad \Box N, \Gamma \Rightarrow N} \quad i=1,2}{\text{cond}_{\Box} \quad \textcolor{blue}{\Box N}, \Gamma \Rightarrow N}$$

where $\Gamma = \Box \vec{N}, \vec{N}$ and we identify the coderivations corresponding to the second and the third premise of the conditional rule, as they only differ on the sub-coderivation $\mathcal{D}_i^*$ ($i = 1$ for the former and $i = 2$ for the latter).

The above coderivation is clearly safe and left-leaning, by the inductive hypotheses for $\mathcal{D}_0, \mathcal{D}_1, \mathcal{D}_2$. To see that it is progressing, note that any infinite branch is either eventually entirely in $\mathcal{D}_0^*, \mathcal{D}_1^*$ or $\mathcal{D}_2^*$, in which case it is progressing by the inductive hypotheses, or it simply loops on $\bullet$ forever, in which case there is a progressing thread along the blue $\textcolor{blue}{\Box N}$.

Moreover, the equational program associated with $\mathcal{D}^*$ is equivalent to:

$$\begin{aligned} f_{\mathcal{D}^*}(0, \vec{x}; \vec{y}) &:= f_{\mathcal{D}_0^*}(\vec{x}; \vec{y}) \\ f_{\mathcal{D}^*}(s_0 x, \vec{x}; \vec{y}) &:= f_{\mathcal{D}_1^*}(x, \vec{x}; \vec{y}, f_{\mathcal{D}^*}(x, \vec{x}; \vec{y})) \text{ if } x \neq 0 \\ f_{\mathcal{D}^*}(s_1 x, \vec{x}; \vec{y}) &:= f_{\mathcal{D}_2^*}(x, \vec{x}; \vec{y}, f_{\mathcal{D}^*}(x, \vec{x}; \vec{y})), \end{aligned}$$

so that $f_{\mathcal{D}}(\vec{x}; \vec{y}) = f_{\mathcal{D}^*}(\vec{x}, \vec{y})$. $\square$

We can also show that CNB is complete for elementary functions by simulating our nested algebra NB. First, we need to introduce the notion of *oracle* for coderivations.⁷

Definition 53 (Oracles for coderivations). Let $\vec{a} = a_1, \dots, a_n$ be a set of safe-normal functions. A $B^-(\vec{a})$ -coderivation is just a usual B^- -coderivation that may use initial sequents of the form $a_i \overline{\Box N^{n_i}, N^{m_i} \Rightarrow N}$, when a_i takes n_i normal and m_i safe inputs. We write:

$$\frac{a_i \overline{\Box N^{n_i}, N^{m_i} \Rightarrow N} \quad i}{\mathcal{D}(\vec{a})} \quad \Gamma \Rightarrow A$$

for a coderivation $\mathcal{D}$ whose initial sequents are among the initial sequents $a_i \overline{\Box N^{n_i}, N^{m_i} \Rightarrow N}$, with $i = 1, \dots, n$. We write $\text{CNB}(\vec{a})$ for the set of CNB-coderivations with initial functions $\vec{a}$. We may sometimes omit indicating some oracles $\vec{a}$ if it is clear from context.

The semantics of such coderivations and the notion of $\text{CNB}(\vec{a})$ -definability are as expected, with coderivations representing functions over the oracles $\vec{a}$, and $f_{\mathcal{D}(\vec{a})} \in \text{CNB}(\vec{a})$ denoting the induced interpretation of $\mathcal{D}(\vec{a})$.

⁷Notice that a similar notational convention discussed in Remark 34 applies *mutatis mutandis* to Definition 53.

Before giving our main completeness result for CNB, we need the following lemma allowing us to ‘pass’ parameters to oracle calls. It is similar to the notion $\mathcal{D}^{\vec{a}}$ from [15, Lemma 42], only we must give a more refined argument due to the unavailability of contraction in our system.

LEMMA 54. Let $\mathcal{D}(a)$ be a regular coderivation over initial sequents $\vec{a}, a$ of the form:

$$\frac{a \overline{\Delta \Rightarrow N} \quad a_i \overline{\Delta_i \Rightarrow N} \quad i}{\mathcal{D}(a)} \quad \square N, .^k., \square N, \Gamma \Rightarrow N$$

where Γ and Δ are lists of non-modal formulas, and the path from the conclusion to each initial sequent a does not contain $\text{cut}_{\square}$ -steps, $\square_I$ -steps and the leftmost premise of a $\text{cond}_{\square}$ -step. Then, there exists an a^* and a regular coderivation $\mathcal{D}^*(a^*)$ with shape:

$$\frac{a^* \overline{\square N, .^k., \square N, \Delta \Rightarrow N} \quad a_i \overline{\Delta_i \Rightarrow N} \quad i}{\mathcal{D}^*(a^*)} \quad \square N, .^k., \square N, \Gamma \Rightarrow N$$

such that:

- $f_{\mathcal{D}^*(a^*)}(\vec{x}; \vec{y}) = f_{\mathcal{D}(a(\vec{x}))}(\vec{x}; \vec{y})$;
- there exists a $\square N$ -thread from the j th $\square N$ in the LHS of the end-sequent to the j th $\square N$ in the context of any occurrence of the initial sequent a^* in $\mathcal{D}^*(a^*)$, for $1 \leq j \leq k$.

Moreover, if $\mathcal{D}(a)$ is progressing, safe or left-leaning, then $\mathcal{D}^*(a^*)$ is also progressing, safe or left-leaning, respectively.

PROOF SKETCH. Let us consider a path B from the root of $\mathcal{D}(a)$ to an occurrence of the initial sequent a . Since the conclusion of $\mathcal{D}(a)$ contains k modal formulas, and B cannot cross $\text{cut}_{\square}$ -steps, B contains exactly k $\square N$ -threads, and all such threads start from the root. Moreover, since a has only non-modal formulas and B cannot cross $\square_I$ -steps or the leftmost premise of a $\text{cond}_{\square}$ -step, we conclude that each of the k $\square N$ -threads must end in the principal formula of a $w_{\square}$ -step. For each j we remove the corresponding $w_{\square}$ -step in B we add an extra $\square N$ to the antecedent of all higher sequents in B (this operation may require us to introduce weakening steps for other branches of the proof). By repeatedly applying the above procedure for each possible path from the root of $\mathcal{D}(a)$ to an initial sequent a we obtain a coderivation with the desired properties. $\square$

Finally, we can give our main simulation result for CNB:

THEOREM 55. If $f(\vec{x}; \vec{y}) \in \text{NB}$ then $f(\vec{x}; \vec{y}) \in \text{CNB}$.

PROOF. We show by induction on $f(\vec{x}; \vec{y}) \in \text{NB}(\vec{a})$ that there is a $\text{CNB}(\vec{a})$ -coderivation $\mathcal{D}_f$ such that:

- (1) $f_{\mathcal{D}_f}(\vec{x}; \vec{y}) = f(\vec{x}; \vec{y})$;
- (2) the path from the conclusion of $\mathcal{D}_f$ to each initial sequent a_i does not contain $\text{cut}_{\square}$ -steps, $\square_I$ -steps and the leftmost premise of a $\text{cond}_{\square}$ -step.

When $f(\vec{x}; \vec{y})$ is an initial function the definition of $\mathcal{D}_f$ is straightforward, as $\vec{a} = \emptyset$ and $\mathcal{D}_f$ can be constructed as in Proposition 6. If $f(\vec{x}; \vec{y}) = a_i(\vec{x}; \vec{y})$ then $\mathcal{D}_f$ is the initial sequent $a_i \overline{\square N, \vec{N} \Rightarrow N}$.

Suppose that $f(\vec{x}; \vec{y}) = h(\vec{x}, g(\vec{x};); \vec{y})$ with $g(\vec{x};) \in \text{NB}(\emptyset)$ and $h(\vec{x}, z; \vec{y}) \in \text{NB}(\emptyset)$. Then, f can be CNB($\emptyset$)-defined by:

$$\frac{\frac{\frac{\emptyset}{\mathcal{D}_g}}{\Box \vec{N} \Rightarrow N} \quad \frac{\frac{\emptyset}{\mathcal{D}_h}}{\Box \vec{N}, \vec{N} \Rightarrow N}}{\frac{\Box_r \Box \vec{N} \Rightarrow \Box N \quad \Box N, \Box \vec{N}, \vec{N} \Rightarrow N}{\text{cut}_{\Box} \Box \vec{N}, \vec{N} \Rightarrow N}}$$

Note that, while we introduce a $\text{cut}_{\Box}$ here, there crucially remains no $\text{cut}_{\Box}$ between the conclusion and an oracle sequent, thanks to the condition that g and h are over no oracles.

Suppose $f(\vec{x}; \vec{y}) = h(\vec{x}; \vec{y}, g(\vec{x}; \vec{y}))$. Then f is CNB($\vec{a}$)-defined by:

$$\frac{\frac{\frac{a_i \overline{\Box \vec{N}, \vec{N} \Rightarrow N}^i}{\mathcal{D}_g}}{\Box \vec{N}, \vec{N} \Rightarrow N} \quad \frac{\frac{a_i \overline{\Box \vec{N}, \vec{N} \Rightarrow N}^i}{\mathcal{D}_h}}{\Box \vec{N}, \vec{N}, N \Rightarrow N}}{\text{cut}_N \Box \vec{N}, \vec{N} \Rightarrow N}$$

Finally, suppose that $f(x, \vec{x}; \vec{y})$ is obtained from $g(\vec{x}; \vec{y})$ over $\emptyset$, and $h_j(a)(x, \vec{x}; \vec{y})$ ($j = 0, 1$) over $a, \vec{a}$ by snrec. Then, $f(0, \vec{x}; \vec{y}) = g(\vec{x}; \vec{y})$ and $f(s_j x, \vec{x}; \vec{y}) = h_j(\lambda \vec{v}. f(x, \vec{x}; \vec{v}))(x, \vec{x}; \vec{y})$. Note that a has same type as $\lambda \vec{v}. f(x, \vec{x}; \vec{v})$, so it is a function taking safe arguments only. Thus, by induction hypothesis, $h_j(a)(x, \vec{x}; \vec{y})$ is CNB($\vec{a}, a$)-defined by:

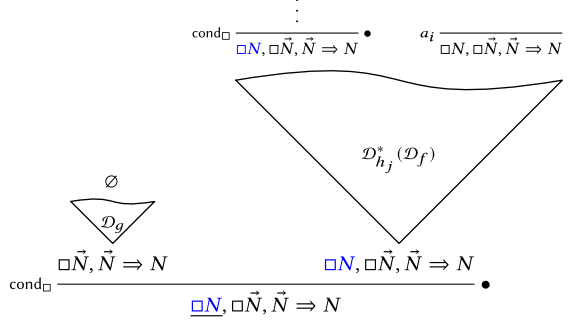
$$\frac{\frac{a \overline{\Box \vec{N} \Rightarrow N}}{\Box \vec{N} \Rightarrow N} \quad \frac{a_i \overline{\Box \vec{N}, \vec{N} \Rightarrow N}^i}{\Box \vec{N}, \vec{N} \Rightarrow N}}{\mathcal{D}_{h_j}(a) \Box N, \Box \vec{N}, \vec{N} \Rightarrow N}$$

where the path from the conclusion of $\mathcal{D}_{h_j}(a)$ to each initial sequent a_i does not contain $\text{cut}_{\Box}$ -steps, $\Box_i$ -steps and the leftmost premise of a $\text{cond}_{\Box}$ -step. By Lemma 54 we obtain the following coderivation:

$$\frac{\frac{a^* \overline{\Box N, \Box \vec{N}, \vec{N} \Rightarrow N}}{\Box N, \Box \vec{N}, \vec{N} \Rightarrow N} \quad \frac{a_i \overline{\Box N, \Box \vec{N}, \vec{N} \Rightarrow N}^i}{\Box N, \Box \vec{N}, \vec{N} \Rightarrow N}}{\mathcal{D}_{h_j}^*(a^*) \Box N, \Box \vec{N}, \vec{N} \Rightarrow N}$$

where $f_{\mathcal{D}_{h_j}^*}(a^*)(x, \vec{x}; \vec{y}) = f_{\mathcal{D}_{h_j}}(a(x, \vec{x}))(x, \vec{x}; \vec{y})$ and there exists a $\Box N$ -thread from the k th modal formula in the antecedent of the conclusion to the k th modal formula in the antecedent of any a^*

initial sequent in $\mathcal{D}^*(a^*)$. We define $\mathcal{D}_f$ as follows:



where we identify the sub-coderivations corresponding to the second and the third premises of the conditional rule. By construction and induction hypothesis, the above coderivation is regular and safe. To see that it is progressing, note that any infinite branch B either hits $\bullet$ infinitely often, in which case there is a progressing thread along the blue $\square N$, by the properties of $\mathcal{D}_{h_j}^*$ inherited from Lemma 54, or B shares a tail with an infinite branch of $\mathcal{D}_g$ or $\mathcal{D}_{h_j}^*(a^*)$, which are progressing by the inductive hypotheses.

We show that $\mathcal{D}_f$ CNB($\vec{a}$)-defines f by induction on x . For the base case, $f_{\mathcal{D}_f}(0, \vec{x}; \vec{y}) = f_{\mathcal{D}_g}(\vec{x}; \vec{y}) = g(\vec{x}; \vec{y}) = f(0, \vec{x}; \vec{y})$. For the inductive step:

$$\begin{aligned} f_{\mathcal{D}_f}(s_j x, \vec{x}; \vec{y}) &= f_{\mathcal{D}_{h_j}^*}(\lambda \vec{v}. f_{\mathcal{D}_f}(x, \vec{x}; \vec{v})) (x, \vec{x}; \vec{y}) \\ &= f_{\mathcal{D}_{h_j}^*}(\lambda \vec{v}. f(x, \vec{x}; \vec{v})) (x, \vec{x}; \vec{y}) \\ &= f_{\mathcal{D}_{h_j}}(\lambda \vec{v}. f(x, \vec{x}; \vec{v})) (x, \vec{x}; \vec{y}) \\ &= h_j(\lambda \vec{v}. f(x, \vec{x}; \vec{v})) (x, \vec{x}; \vec{y}) \\ &= f(s_j x, \vec{x}; \vec{y}). \end{aligned}$$

This completes the proof. $\square$

6.2 The Translation Lemma

The goal of this section is to prove the following theorem, which shows a translation of CNB-coderivations into functions of NB^C mapping, in particular, CB-derivations into functions of B^C :

THEOREM 56 (TRANSLATION LEMMA). *Let $\mathcal{D}$ be a CNB-coderivation. Then, there exists a set of n functions $(f_i)_{1 \leq i \leq n}$ such that $f_1 = f_{\mathcal{D}}$ and, for all i :*

$$f_i(\vec{x}; \vec{y}) = h_i(\lambda \vec{u} \subset \vec{x}, \lambda \vec{v}. f_j(\vec{u}; \vec{v}))_{1 \leq j \leq n}(\vec{x}; \vec{y}), \quad (16)$$

where $h_i \in \text{NB}^C(a_i)_{1 \leq i \leq n}$. Moreover, if $\mathcal{D}$ is a CB-coderivation then, for all i :

$$f_i(\vec{x}; \vec{y}) = h_i(\lambda \vec{u} \subset \vec{x}, \lambda \vec{v} \subseteq \vec{y}. f_j(\vec{u}; \vec{v}))_{1 \leq j \leq n}(\vec{x}; \vec{y}) \quad (17)$$

and $h_i \in \text{B}^C(a_i)_{1 \leq i \leq n}$.

This would conclude our characterisation of CB and CNB in terms of computational complexity.

A formal proof of the translation lemma requires a more general statement (i.e., Lemma 61) as well as a few technical notions and intermediate results. First, we observe that a regular coderivation can be naturally seen as a finite tree with ‘backpointers’, a representation known as *cycle normal form*, cf. [9, 10].

Definition 57 (Cycle Normal Form). Let $\mathcal{D}$ be a regular B^- -coderivation. The *cycle normal form* (or simply *cycle nf*) of $\mathcal{D}$ is a pair $\langle \mathcal{D}, R_{\mathcal{D}} \rangle$, where $R_{\mathcal{D}}$ is a partial self-mapping on the nodes of $\mathcal{D}$ whose domain of definition is denoted $Bud(\mathcal{D})$ and:

- (i) every infinite branch of $\mathcal{D}$ contains some (unique) $v \in Bud(\mathcal{D})$;
- (ii) if $v \in Bud(\mathcal{D})$ then both $R_{\mathcal{D}}(v) \sqsubset v$ and $\mathcal{D}_{R_{\mathcal{D}}(v)} = \mathcal{D}_v$;
- (iii) for any two distinct nodes $\mu \sqsubset v$ strictly below $Bud(\mathcal{D})$, $\mathcal{D}_{\mu} \neq \mathcal{D}_v$

We call any $v \in Bud(\mathcal{D})$ a *bud*, and $R_{\mathcal{D}}(v)$ its *companion*. A *terminal* node is either a leaf of $\mathcal{D}$ or a bud. The set of nodes of $\mathcal{D}$ bounded above by a terminal node is denoted $T_{\mathcal{D}}$. Given a node $v \in T_{\mathcal{D}}$, we define $Bud_v(\mathcal{D})$ as the restriction of buds to those above v .

Remark 58. The cycle normal form of a regular coderivation $\mathcal{D}$ always exists, as by definition any infinite branch contains a node v such that $\mathcal{D}_v = \mathcal{D}_{\mu}$ for some node μ below v . $Bud(\mathcal{D})$ is designed to consist of just the *least* such nodes, so that by construction the cycle normal form is unique. Note that $Bud(\mathcal{D})$ must form an $\sqsubset$ -antichain: if $\mu, v \in Bud(\mathcal{D})$ with $\mu \sqsubset v$, then $R_{\mathcal{D}}(\mu) \sqsubset \mu$ are below $Bud(\mathcal{D})$ but we have $\mathcal{D}_{R_{\mathcal{D}}(\mu)} = \mathcal{D}_{\mu}$ by point (ii) above, contradicting point (iii).

Also, notice that any branch of $\mathcal{D}$ contains a leaf of $T_{\mathcal{D}}$. Moreover, since $Bud(\mathcal{D})$ is an antichain, the leaves of $T_{\mathcal{D}}$ defines a ‘bar’ across $\mathcal{D}$, and so $T_{\mathcal{D}}$ is a finite tree.

The following proposition allows us to reformulate progressiveness, safety and left-leaning conditions for cycle normal forms.

PROPOSITION 59. *Let $\mathcal{D}$ be a regular B^- -coderivation with cycle nf $\langle \mathcal{D}, R_{\mathcal{D}} \rangle$. For any $v \in Bud(\mathcal{D})$, the (finite) path π from $R_{\mathcal{D}}(v)$ to v satisfies:*

- (1) if $\mathcal{D}$ is progressing, π must contain the conclusion of an instance of $\text{cond}_{\square}$;
- (2) if $\mathcal{D}$ is a CNB-coderivation, π cannot contain the conclusion of $\text{cut}_{\square}$, $\square_I$, $w_{\square}$, and the leftmost premise of $\text{cond}_{\square}$;
- (3) if $\mathcal{D}$ is a CB-coderivation, π cannot contain the conclusion of w_N , the leftmost premise of cond_N , and the rightmost premise of cut_N .

PROOF. By definition of cycle nf, each path from $R_{\mathcal{D}}(v)$ to v in $\langle \mathcal{D}, R_{\mathcal{D}} \rangle$ is contained in a branch of $\mathcal{D}$ such that each rule instance in the former appears infinitely many times in the latter. Hence:

- (i) if $\mathcal{D}$ is progressing, the path contains the conclusion of an instance of $\text{cond}_{\square}$;
- (ii) if $\mathcal{D}$ is safe, the path cannot contain the conclusion of a $\text{cut}_{\square}$ rule;
- (iii) if $\mathcal{D}$ is left-leaning, the path cannot contain the rightmost premise of a cut_N rule.

This shows point 1. Let us consider point 2. By point (ii), if $\mathcal{D}$ is safe then, going from a node μ of the path to each of its children μ' , the number of modal formulas in the context of the corresponding sequents cannot increase. Moreover, the only cases where this number strictly decreases is when μ is the conclusion of $\square_I$, $w_{\square N}$, or when μ' is the leftmost premise of $\text{cond}_{\square}$. Since $R_{\mathcal{D}}(v)$ and v must be labelled with the same sequent, all such cases are impossible. As for point 3 we notice that, by point (iii) and the above reasoning, if $\mathcal{D}$ is safe and left-leaning then, going from a node μ of the path to each of its children μ' , the number of non-modal formulas in the context of the corresponding sequents cannot increase. Moreover, the only cases where this number strictly decreases is when μ is the conclusion of w_N , or when μ' is the leftmost premise of cond_N . Since $R_{\mathcal{D}}(v)$ and v must be labelled with the same sequent, all such cases are impossible. $\square$

In what follows, we shall indicate circularities in cycle nfs explicitly by extending both CNB and CB with a new inference rule called *dis*:

$$\text{dis} \frac{\Gamma \Rightarrow A}{\Gamma \Rightarrow A} X,$$

where X is a finite set of nodes. In this presentation, we insist that each companion v of the cycle nf $\langle \mathcal{D}, R_{\mathcal{D}} \rangle$ is always the conclusion of an instance of *dis*, where X denotes the set of buds v' such that $R_{\mathcal{D}}(v') = v$. This expedient will allow us to treat the case of companions separately, avoiding repeating the same argument for each rule.

To facilitate the translation, we shall define two disjoint sets C_v and O_v . Intuitively, given a cycle nf $\langle \mathcal{D}, R_{\mathcal{D}} \rangle$ and $v \in T_{\mathcal{D}}$, C_v is the set of companions above v , while O_v is the set of buds whose companion is strictly below v .

Definition 60. Let $\langle \mathcal{D}, R_{\mathcal{D}} \rangle$ be the cycle normal form of a B^- -coderivation $\mathcal{D}$. We define the following two sets for any $v \in T_{\mathcal{D}}$:

$$\begin{aligned} C_v &:= \{\mu \in R_{\mathcal{D}}(\text{Bud}_v(\mathcal{D})) \mid v \sqsubseteq \mu\} \\ O_v &:= \{\mu \in \text{Bud}_v(\mathcal{D}) \mid R_{\mathcal{D}}(\mu) \sqsubset v\}. \end{aligned}$$

We now state a generalised version of the translation lemma and prove it in detail. The idea, here, is to associate with each node $v_0 \in T_{\mathcal{D}}$ an instance of the scheme ssnrec_C such that:

- the scheme simultaneously defines the functions in $C_{v_0} = \{f_v \mid v \in C_{v_0} \cup \{v_0\}\}$, with the help of an additional set of oracles $O_{v_0} = \{a_\mu \mid \mu \in O_{v_0}\}$
- if $O_{v_0} = \emptyset$ then, for every $f_v \in C_{v_0}$ it holds that $f_v = f_{\mathcal{D}_v}$, i.e., f_v defines the interpretation of $\mathcal{D}_v$ via a set of equations (see Definition 10).

In particular, when v_0 is the root of $\mathcal{D}$ then $O_{v_0} = \emptyset$, and so $f_{v_0} = f_{\mathcal{D}_{v_0}} = f_{\mathcal{D}}$. Moreover, since f_{v_0} is defined by an instance of ssnrec_C then $f_{\mathcal{D}} \in \text{NB}^C$ by Proposition 40. As a special case, if $\mathcal{D}$ is a CB-coderivation then f_{v_0} can be defined by an instance of ssrec_C , and so $f_{\mathcal{D}} \in B^C$.

LEMMA 61 (TRANSLATION LEMMA, GENERAL VERSION). *If $\langle \mathcal{D}, R_{\mathcal{D}} \rangle$ is the cycle nf of a CNB-coderivation $\mathcal{D}$ and $v_0 \in T_{\mathcal{D}}$, then $\forall v \in C_{v_0} \cup \{v_0\}$:*

$$f_v(\vec{x}; \vec{y}) = h_v \left((\lambda \vec{u} \subseteq \vec{x}, \lambda \vec{v}. f_\mu(\vec{u}; \vec{v}))_{\mu \in C_{v_0}}, (\lambda \vec{u} \subseteq \vec{x}, \lambda \vec{v}. a_\mu(\vec{u}; \vec{v}))_{\mu \in O_{v_0}} \right) (\vec{x}; \vec{y}),$$

where:

- (1) $h_v \in \text{NB}^C((f_\mu)_{\mu \in C_{v_0}}, (a_\mu)_{\mu \in O_{v_0}})$, and so $f_v \in \text{NB}^C((a_\mu)_{\mu \in O_{v_0}})$;
- (2) for all $\mu \in O_{v_0}$, the order $\vec{u} \subseteq \vec{x}$ in $\lambda \vec{u} \subseteq \vec{x}, \lambda \vec{v}. a_\mu(\vec{u}; \vec{v})$ is strict if the path from v to μ in $T_{\mathcal{D}}$ contains the conclusion of an instance of $\text{cond}_{\square}$;
- (3) if moreover $\mathcal{D}$ is a CB-coderivation then

$$f_v(\vec{x}; \vec{y}) = h_v \left((\lambda \vec{u} \sqsubset \vec{x}, \lambda \vec{v} \subseteq \vec{y}. f_\mu(\vec{u}; \vec{v}))_{\mu \in C_{v_0}}, (\lambda \vec{u} \subseteq \vec{x}, \lambda \vec{v} \subseteq \vec{y}. a_\mu(\vec{u}; \vec{v}))_{\mu \in O_{v_0}} \right) (\vec{x}; \vec{y}),$$

with $h_v \in B^C((f_\mu)_{\mu \in C_{v_0}}, (a_\mu)_{\mu \in O_{v_0}})$, and so $f_v \in B^C((a_\mu)_{\mu \in O_{v_0}})$.

PROOF. By induction on the longest distance of v_0 from a leaf of $T_{\mathcal{D}}$. Concerning the cases where v_0 is the conclusion of an instance of *id* or *0* we have $C_{v_0} = O_{v_0} = \emptyset$, and we simply set $h_{v_0} := \text{id}$ and $h_{v_0} := 0$, respectively. If v_0 is the conclusion of a bud then $C_{v_0} = \emptyset$ and $O_{v_0} = \{v_0\}$, and we simply set $h_{v_0} := a_{v_0}$. The cases where v is the conclusion of a rule in $\{w_N, w_{\square}, \square_l, e_N, e_{\square}, \square_r, s_0, s_1\}$ are straightforward. Notice that for the cases $w_N, w_{\square}, \square_l$ we have $O_{v_0} = \emptyset$ by Proposition 59 points 2 and 3.

Let us now consider the case where v_0 is the conclusion of an instance of $\text{cut}_\square$ with premises v_1 and v_2 . By Proposition 59.2 we have $O_{v_0} = \emptyset$. By induction hypothesis we have $f_{v_1}(\vec{x}; \vec{y}), f_{v_2}(\vec{x}; x; \vec{y}) \in \text{NB}^\subset$. Since the conclusion of $\mathcal{D}_{v_1}$ has modal succedent, by Proposition 18 there must be a coderivation $\mathcal{D}^*$ such that $f_{\mathcal{D}^*}(\vec{x};) = f_{v_1}(\vec{x}; \vec{y}) \in \text{NB}^\subset$. Moreover, by Proposition 29, if $\mathcal{D}_{v_1}$ is a CB-coderivation then so too is $\mathcal{D}^*$. Hence, we define $h_{v_0}(\vec{x}; \vec{y}) := f_{v_2}(\vec{x}, f_{\mathcal{D}^*}(\vec{x};); \vec{y})$. It is easy to see that points 1–3 hold.

Suppose now that v_0 is the conclusion of an instance of cut_N with premises v_1 and v_2 . Then, $O_{v_0} = O_{v_1} \cup O_{v_2}$ and $C_{v_0} = C_{v_1} \cup C_{v_2}$. By induction hypothesis on v_1 and v_2 we have:

$$\begin{aligned} f_{v_1}(\vec{x}; \vec{y}) &= h_{v_1} \left((\lambda \vec{u} \subset \vec{x}, \lambda \vec{v}. f_\mu(\vec{u}; \vec{v}))_{\mu \in C_{v_1}}, (\lambda \vec{u} \subseteq \vec{x}, \lambda \vec{v}. a_\mu(\vec{u}; \vec{v}))_{\mu \in O_{v_1}} \right) (\vec{x}; \vec{y}) \\ f_{v_2}(\vec{x}; y, \vec{y}) &= h_{v_2} \left((\lambda \vec{u} \subset \vec{x}, \lambda v, \vec{v}. f_\mu(\vec{u}; v, \vec{v}))_{\mu \in C_{v_2}}, (\lambda \vec{u} \subseteq \vec{x}, \lambda v, \vec{v}. a_\mu(\vec{u}; v, \vec{v}))_{\mu \in O_{v_2}} \right) (\vec{x}; y, \vec{y}). \end{aligned}$$

So that we set

$$h_{v_0}(\vec{x}; \vec{y}) := f_{v_2}(\vec{x}; f_{v_1}(\vec{x}; \vec{y}), \vec{y}).$$

Points 1 and 2 hold by applying the induction hypothesis. Concerning point 3, notice that if $\mathcal{D}$ is a CB-coderivation then $O_{v_2} = \emptyset$ by Proposition 59.3. By applying the induction hypothesis on v_1 and v_2 , we have $f_{v_2}(\vec{x}; y, \vec{y}) \in \text{B}^\subset$ and $f_{v_1}(\vec{x}; \vec{y}) = h_{v_1}((\lambda \vec{u} \subset \vec{x}, \lambda \vec{v} \subseteq \vec{y}. f_\mu(\vec{u}; \vec{v}))_{\mu \in C_{v_1}}, (\lambda \vec{u} \subseteq \vec{x}, \lambda \vec{v} \subseteq \vec{y}. a_\mu(\vec{u}; \vec{v}))_{\mu \in O_{v_1}})(\vec{x}; \vec{y})$, so that:

$$f_{v_0}(\vec{x}; \vec{y}) = h_{v_0} \left((\lambda \vec{u} \subset \vec{x}, \lambda \vec{v} \subseteq \vec{y}. f_\mu(\vec{u}; \vec{v}))_{\mu \in C_{v_0}}, (\lambda \vec{u} \subseteq \vec{x}, \lambda \vec{v} \subseteq \vec{y}. a_\mu(\vec{u}; \vec{v}))_{\mu \in O_{v_0}} \right) (\vec{x}; \vec{y}).$$

Suppose that v_0 is the conclusion of a $\text{cond}_\square$ step with premises v' , v_1 , and v_2 . By Proposition 59.2, we have $O_{v'} = \emptyset$, so that $f_{v'} \in \text{NB}^\subset$ (resp. $f_{v'} \in \text{B}^\subset$) by induction hypothesis. By definition, $O_{v_0} = O_{v_1} \cup O_{v_2}$ and $C_{v_0} = C_{v_1} \cup C_{v_2}$. Then, we set:

$$h_{v_0}(x, \vec{x}; \vec{y}) = \text{cond}(:, x, f_{v'}(\vec{x}; \vec{y}), f_{v_1}(\mathbf{p}(x;), \vec{x}; \vec{y}), f_{v_2}(\mathbf{p}(x;), \vec{x}; \vec{y})),$$

where $\mathbf{p}(x;)$ can be defined from $\mathbf{p}(: x)$ and projections. By induction hypothesis on v_i :

$$\begin{aligned} f_{v_i}(\mathbf{p}(x;), \vec{x}; \vec{y}) &= h_{v_i} \left((\lambda u, \vec{u} \subset \mathbf{p}(x;), \vec{x}, \lambda \vec{v}. f_\mu(u, \vec{u}; \vec{v}))_{\mu \in C_{v_i}}, (\lambda u, \vec{u} \subseteq \mathbf{p}(x;), \vec{x}, \lambda \vec{v}. a_\mu(u, \vec{u}; \vec{v}))_{\mu \in O_{v_i}} \right) (\mathbf{p}(x;), \vec{x}; \vec{y}) \\ &= h_{v_i} \left((\lambda u, \vec{u} \subset x, \vec{x}, \lambda \vec{v}. f_\mu(u, \vec{u}; \vec{v}))_{\mu \in C_{v_i}}, (\lambda u, \vec{u} \subset x, \vec{x}, \lambda \vec{v}. a_\mu(u, \vec{u}; \vec{v}))_{\mu \in O_{v_i}} \right) (\mathbf{p}(x;), \vec{x}; \vec{y}), \end{aligned}$$

hence:

$$f_{v_0}(x, \vec{x}; \vec{y}) = h_{v_0} \left((\lambda u, \vec{u} \subset x, \vec{x}, \lambda \vec{v}. f_\mu(u, \vec{u}; \vec{v}))_{\mu \in C_{v_0}}, (\lambda u, \vec{u} \subset x, \vec{x}, \lambda \vec{v}. a_\mu(u, \vec{u}; \vec{v}))_{\mu \in O_{v_0}} \right) (x, \vec{x}; \vec{y}),$$

which shows in particular point 2. Points 1 and 3 are straightforward.

Let us now consider the case where v_0 is an instance of cond_N . By Proposition 59.3 we have $O_{v'} = \emptyset$, so that $f_{v'} \in \text{NB}^\subset$ (resp., $f_{v'} \in \text{B}^\subset$) by induction hypothesis. By definition, $O_{v_0} = O_{v_1} \cup O_{v_2}$ and $C_{v_0} = C_{v_1} \cup C_{v_2}$. Then, we set:

$$h_{v_0}(\vec{x}; y, \vec{y}) = \text{cond}(:, y, f_{v'}(\vec{x}; \vec{y}), f_{v_1}(\vec{x}; \mathbf{p}(: y), \vec{y}), f_{v_2}(\vec{x}; \mathbf{p}(: y), \vec{y})).$$

By induction hypothesis on v_i :

$$f_{v_i}(\vec{x}; \mathbf{p}(: y), \vec{y}) = h_{v_i} \left((\lambda \vec{u} \subset x, \vec{x}, \lambda v, \vec{v}. f_\mu(\vec{u}; v, \vec{v}))_{\mu \in C_{v_i}}, (\lambda \vec{u} \subseteq x, \vec{x}, \lambda v, \vec{v}. a_\mu(\vec{u}; v, \vec{v}))_{\mu \in O_{v_i}} \right) (\vec{x}; \mathbf{p}(: y), \vec{y}),$$

and hence:

$$f_{v_0}(\vec{x}; y, \vec{y}) = h_{v_0} \left((\lambda \vec{u} \subset \vec{x}, \lambda v, \vec{v}. f_\mu(\vec{u}; v, \vec{v}))_{\mu \in C_{v_0}}, (\lambda \vec{u} \subseteq \vec{x}, \lambda v, \vec{v}. a_\mu(\vec{u}; v, \vec{v}))_{\mu \in O_{v_0}} \right) (\vec{x}; y, \vec{y})$$

Points 1 and 2 are straightforward. Concerning point 3 notice that by induction hypothesis on v_i we have:

$$f_{v_i}(\vec{x}; p(; y), \vec{y}) = h_{v_i} \left((\lambda \vec{u} \subset x, \vec{x}, \lambda v, \vec{v} \subseteq p(; y), \vec{y}.f_\mu(\vec{u}; v, \vec{v}))_{\mu \in C_{v_i}}, (\lambda \vec{u} \subseteq x, \vec{x}, \lambda v, \vec{v} \subseteq p(; y), \vec{y}.a_\mu(\vec{u}; v, \vec{v}))_{\mu \in O_{v_i}} \right) (\vec{x}; p(; y), \vec{y}),$$

and so we have:

$$f_{v_0}(\vec{x}; y, \vec{y}) = h_{v_0} \left((\lambda \vec{u} \subset \vec{x}, \lambda v, \vec{v} \subset y, \vec{y}.f_\mu(\vec{u}; v, \vec{v}))_{\mu \in C_{v_0}}, (\lambda \vec{u} \subseteq \vec{x}, \lambda v, \vec{v} \subset y, \vec{y}.a_\mu(\vec{u}; v, \vec{v}))_{\mu \in O_{v_0}} \right) (\vec{x}; y, \vec{y}).$$

Let us finally consider the case where v_0 is the conclusion of an instance of *dis* with premise v' , where X is the set of buds labelling the rule. We have $O_{v_0} = O_{v'} - X$ and $C_{v_0} = C_{v'} \cup \{v_0\}$. We want to find $(h_v)_{v \in C_{v'} \cup \{v_0\}}$ defining the equations for $(f_v)_{v \in C_{v'} \cup \{v_0\}}$ in such a way that points 1–3 hold. First, note that, by definition of cycle *nf*, $f_{\mathcal{D}_{v_0}}(\vec{x}; \vec{y}) = f_{\mathcal{D}_{v'}}(\vec{x}; \vec{y}) = f_{\mathcal{D}_\mu}(\vec{x}; \vec{y})$ for all $\mu \in X$. Since $O_{v'} = O_{v_0} \cup X$ and the path from v' to any $\mu \in X$ must cross an instance of *cond*_□ by Proposition 59.1, by induction hypothesis on v' there exists a family $(h'_v)_{v \in C_{v'} \cup \{v'\}}$ such that:

$$f_{v'}(\vec{x}; \vec{y}) = h'_{v'} \left((\lambda \vec{u} \subset \vec{x}, \lambda \vec{v}.f_\mu(\vec{u}; \vec{v}))_{\mu \in C_{v'}}, (\lambda \vec{u} \subseteq \vec{x}, \lambda \vec{v}.a_\mu(\vec{u}; \vec{v}))_{\mu \in O_{v_0}}, (\lambda \vec{u} \subset \vec{x}, \lambda \vec{v}.a_\mu(\vec{u}; \vec{v}))_{\mu \in X} \right) (\vec{x}; \vec{y}), \quad (18)$$

and for all $v \in C_{v'}$:

$$f_v(\vec{x}; \vec{y}) = h'_v \left((\lambda \vec{u} \subset \vec{x}, \lambda \vec{v}.f_\mu(\vec{u}; \vec{v}))_{\mu \in C_{v'}}, (\lambda \vec{u} \subseteq \vec{x}, \lambda \vec{v}.a_\mu(\vec{u}; \vec{v}))_{\mu \in O_{v_0}}, (\lambda \vec{u} \subseteq \vec{x}, \lambda \vec{v}.a_\mu(\vec{u}; \vec{v}))_{\mu \in X} \right) (\vec{x}; \vec{y}), \quad (19)$$

we fix the oracles as $a_\mu := f_\mu$ for all $\mu \in X$ and set $h_{v_0} := h'_{v'}$ so that:

$$\begin{aligned} f_{v_0}(\vec{x}; \vec{y}) &= h_{v_0} \left((\lambda \vec{u} \subset \vec{x}, \lambda \vec{v}.f_\mu(\vec{u}; \vec{v}))_{\mu \in C_{v'}}, (\lambda \vec{u} \subseteq \vec{x}, \lambda \vec{v}.a_\mu(\vec{u}; \vec{v}))_{\mu \in O_{v_0}}, \lambda \vec{u} \subset \vec{x}, \lambda \vec{v}.f_{v_0}(\vec{u}; \vec{v}) \right) (\vec{x}; \vec{y}) \\ &= h_{v_0} \left((\lambda \vec{u} \subset \vec{x}, \lambda \vec{v}.f_\mu(\vec{u}; \vec{v}))_{\mu \in C_{v'} \cup \{v_0\}}, (\lambda \vec{u} \subseteq \vec{x}, \lambda \vec{v}.a_\mu(\vec{u}; \vec{v}))_{\mu \in O_{v_0}} \right) (\vec{x}; \vec{y}) \\ &= h_{v_0} \left((\lambda \vec{u} \subset \vec{x}, \lambda \vec{v}.f_\mu(\vec{u}; \vec{v}))_{\mu \in C_{v_0}}, (\lambda \vec{u} \subseteq \vec{x}, \lambda \vec{v}.a_\mu(\vec{u}; \vec{v}))_{\mu \in O_{v_0}} \right) (\vec{x}; \vec{y}), \end{aligned} \quad (20)$$

and for all $v \in C_{v'}$:

$$f_v(\vec{x}; \vec{y}) = h'_v \left((\lambda \vec{u} \subset \vec{x}, \lambda \vec{v}.f_\mu(\vec{u}; \vec{v}))_{\mu \in C_{v'}}, (\lambda \vec{u} \subseteq \vec{x}, \lambda \vec{v}.a_\mu(\vec{u}; \vec{v}))_{\mu \in O_{v_0}}, \lambda \vec{u} \subseteq \vec{x}, \lambda \vec{v}.f_{v_0}(\vec{u}; \vec{v}) \right) (\vec{x}; \vec{y}). \quad (21)$$

Notice that $\vec{u} \subseteq \vec{x}$ might not be strict in $\lambda \vec{u} \subseteq \vec{x}, \lambda \vec{v}.f_{v_0}(\vec{u}; \vec{v})$. However, by Equation (20) f_{v_0} can be rewritten to $h_{v_0} \left((\lambda \vec{u} \subset \vec{x}, \lambda \vec{v}.f_\mu(\vec{u}; \vec{v}))_{\mu \in C_{v_0}}, (\lambda \vec{u} \subseteq \vec{x}, \lambda \vec{v}.a_\mu(\vec{u}; \vec{v}))_{\mu \in O_{v_0}} \right)$, and so h'_v can be rewritten to a h_v^* such that:

$$f_v(\vec{x}; \vec{y}) = h_v^* \left((\lambda \vec{u} \subset \vec{x}, \lambda \vec{v}.f_\mu(\vec{u}; \vec{v}))_{\mu \in C_{v_0}}, (\lambda \vec{u} \subseteq \vec{x}, \lambda \vec{v}.a_\mu(\vec{u}; \vec{v}))_{\mu \in O_{v_0}} \right) (\vec{x}; \vec{y}), \quad (22)$$

we set $h_v := h_v^*$. Points 1 and 2 are straightforward. Point 3 follows by noticing that the construction does not affect the safe arguments. $\square$

Finally, we can establish the main result of this article:

COROLLARY 62. *We have the following:*

- $f(\vec{x};) \in \text{CB}$ if and only if $f(\vec{x}) \in \text{FPTIME}$;
- $f(\vec{x};) \in \text{CNB}$ if and only if $f(\vec{x}) \in \text{FELEMENTARY}$.

PROOF SKETCH. Soundness ($\Rightarrow$) follows from Theorem 56, by showing that B^C and NB^C are closed under *simultaneous* versions of their recursion schemes. Completeness ($\Leftarrow$) follows from Theorem 52 and Theorem 55. $\square$

7 Conclusions and Further Remarks

In this work we presented two-tiered circular type systems CB and CNB and showed that they capture polynomial-time and elementary computation, respectively. This is the first time that methods of circular proof theory have been applied in ICC. Along the way we gave novel relativised algebras for these classes based on safe (nested) recursion on well-founded relations.

Since the conference version [11] of this work was published, other works in Cyclic Implicit Complexity have appeared, building on the present work, in particular in the setting of *non-uniform computation* [1, 12].

7.1 Unary Notation and Linear Space

It is well-known that FLINSPACE, i.e., the class of functions computable in linear space, can be captured by reformulating B in *unary* notation (see [4]). A similar result can be obtained for CB by just defining a unary version of the conditional in B^C (similarly to the ones in [15, 25]) and by adapting the proofs of Lemma 41, Theorem 52 and Theorem 56. On the other hand, CNB is (unsurprisingly) not sensitive to such choice of notation.

7.2 On Unnested Recursion with Compositions

We believe that Lemma 41 can be adapted to establish a polynomial bound on the growth rate for the function algebra extending B^- by ‘Safe Composition During Safe Recursion’, cf. Remark 36. We conjecture that this function algebra and its extension to recursion on permutation of prefixes (cf. Definition 38) capture precisely the class FSPACE. Indeed, several function algebras for FSPACE have been proposed in the literature, many of which involve variants of (5) (see [28, 32]). These recursion schemes reflect the parallel nature of polynomial space functions, which can be defined in terms of alternating polynomial time computation. We suspect that a circular proof theoretic characterisation of this class can thus be achieved by extending CB with a ‘parallel’ version of the cut rule and by adapting the left-leaning criterion appropriately. Parallel cuts might also play a fundamental role for potential circular proof theoretic characterisations of circuit complexity classes, like ALOGTIME or NC.

7.3 Towards Higher-Order Cyclic Implicit Complexity

It would be pertinent to pursue higher-order versions of both CNB and CB, in light of precursory works [15, 25] in circular proof theory as well as ICC [7, 22, 27]. In the case of polynomial-time, for instance, a soundness result for some higher-order version of CB might follow by translation to (a sequent-style formulation of) Hofmann’s SLR [22]. Analogous translations might be defined for a higher-order version of CNB once the linearity restrictions on the recursion operator of SLR are dropped. Finally, as SLR is essentially a subsystem of Gödel’s system T, such translations could refine the results on the abstraction complexity i.e., type level) of the circular version of system T in [13, 15].

Acknowledgements

We would like to thank the anonymous reviewers for their valuable comments and suggestions. We are also indebted to Dominik Wehr for pointing out a minor error in the conference version of this article [11] (see Remark 36).

References

- [1] Matteo Acclavio, Gianluca Curzi, and Giulio Guerrieri. 2024. Infinitary cut-elimination via finite approximations. In *Proceedings of the 32nd EACSL Annual Conference on Computer Science Logic (CSL '24)*, Aniello Murano and Alexandra Silva (Eds.), LIPIcs, Vol. 288, Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 8:1–8:19. DOI: <https://doi.org/10.4230/LIPICS.CSL.2024.8>
- [2] David Baelde, Amina Doumane, Denis Kuperberg, and Alexis Saurin. 2020. Bouncing threads for infinitary and circular proofs. arXiv:2005.08257. Retrieved from <https://arxiv.org/abs/2005.08257>
- [3] David Baelde, Amina Doumane, and Alexis Saurin. 2016. Infinitary proof theory: The multiplicative additive case. In *Proceedings of the 25th EACSL Annual Conference on Computer Science Logic*, Vol. 62, 42:1–42:17. DOI: <https://doi.org/10.4230/LIPICS.CSL.2016.42>
- [4] Stephen Bellantoni. 1992. Predicative Recursion and Computational Complexity. PhD thesis.
- [5] Stephen Bellantoni. 1995. Predicative recursion and the polytime hierarchy. In *Feasible Mathematics II*. Peter Clote and Jeffrey B. Remmel (Eds.), Birkhäuser Boston, 15–29.
- [6] Stephen Bellantoni and Stephen Cook. 1992. A new recursion-theoretic characterization of the polytime functions (extended abstract). In *Proceedings of the 24th Annual ACM Symposium on Theory of Computing (STOC '92)*. ACM, New York, NY, 283–293. DOI: <https://doi.org/10.1145/129712.129740>
- [7] Stephen J. Bellantoni, Karl-Heinz Niggl, and Helmut Schwichtenberg. 2000. Higher type recursion, ramification and polynomial time. *Annals of Pure and Applied Logic* 104, 1–3 (2000), 17–30. DOI: [https://doi.org/10.1016/S0168-0072\(00\)00006-3](https://doi.org/10.1016/S0168-0072(00)00006-3)
- [8] Stefano Berardi and Makoto Tatsuta. 2017. Equivalence of inductive definitions and cyclic proofs under arithmetic. In *Proceedings of the 32nd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS '17)*. IEEE Computer Society, 1–12. DOI: <https://doi.org/10.1109/LICS.2017.8005114>
- [9] James Brotherston. 2005. Cyclic proofs for first-order logic with inductive definitions. In *Proceedings of the International Conference on Automated Reasoning with Analytic Tableaux and Related Methods (TABLEAUX '05)*, Bernhard Beckert (Ed.), *Lecture Notes in Computer Science*, Vol. 3702, Springer, 78–92. DOI: https://doi.org/10.1007/11554554_8
- [10] James Brotherston and Alex Simpson. 2011. Sequent calculi for induction and infinite descent. *Journal of Logic and Computation* 21, 6 (2011), 1177–1216.
- [11] Gianluca Curzi and Anupam Das. 2022. Cyclic implicit complexity. In *Proceedings of the 37th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS '22)*, Christel Baier and Dana Fisman (Eds.), ACM, 19:1–19:13. DOI: <https://doi.org/10.1145/3531130.3533340>
- [12] Anupam Das and Gianluca Curzi. 2023. Non-uniform complexity via non-wellfounded proofs. In *Proceedings of the 31st EACSL Annual Conference on Computer Science Logic (CSL '23)*, Bartek Klin and Elaine Pimentel (Eds.), LIPIcs, Vol. 252, Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 16:1–16:18. DOI: <https://doi.org/10.4230/LIPICS.CSL.2023.16>
- [13] Das Anupam. 2020. A circular version of Gödel's T and its abstraction complexity. arXiv:2012.14421. Retrieved from <https://arxiv.org/abs/2012.14421>
- [14] Anupam Das. 2020. On the logical complexity of cyclic arithmetic. *Logical Methods in Computer Science* 16, 1 (2020), 1–39. DOI: [https://doi.org/10.23638/LMCS-16\(1:1\)2020](https://doi.org/10.23638/LMCS-16(1:1)2020)
- [15] Anupam Das. 2021. On the logical strength of confluence and normalisation for cyclic proofs. In *Proceedings of the 6th International Conference on Formal Structures for Computation and Deduction (FSCD '21)*, Naoki Kobayashi (Ed.), LIPIcs, Vol. 195, Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 29:1–29:23. DOI: <https://doi.org/10.4230/LIPICS.FSCD.2021.29>
- [16] Anupam Das and Damien Pous. 2017. A cut-free cyclic proof system for kleene algebra. In *Proceedings of the International Conference on Automated Reasoning with Analytic Tableaux and Related Methods*. Springer, 261–277.
- [17] Anupam Das and Damien Pous. 2018. Non-wellfounded proof theory for (kleene+action)(algebras+lattices). In *Proceedings of the 27th EACSL Annual Conference on Computer Science Logic (CSL '18)*, Dan Ghica and Achim Jung (Eds.), Leibniz International Proceedings in Informatics (LIPIcs), Vol. 119, Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 19:1–19:18. DOI: <https://doi.org/10.4230/LIPICS.CSL.2018.19>
- [18] Christian Dax, Martin Hofmann, and Martin Lange. 2006. A proof system for the linear time μ -calculus. In *Proceedings of the 26th International Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS '06)*, S. Arun-Kumar and Naveen Garg (Eds.), *Lecture Notes in Computer Science*, Vol. 4337, Springer, 273–284. DOI: https://doi.org/10.1007/11944836_26
- [19] Christian Dax, Martin Hofmann, and Martin Lange. 2006. A proof system for the linear time μ -calculus. In *Proceedings of the International Conference on Foundations of Software Technology and Theoretical Computer Science*. Springer, 273–284.
- [20] Abhishek De and Alexis Saurin. 2019. Infinites: The parallel syntax for non-wellfounded proof-theory. In *Proceedings of the 28th International Conference on Automated Reasoning with Analytic Tableaux and Related Methods (TABLEAUX*

- '19), Serenella Cerrito and Andrei Popescu (Eds.), Lecture Notes in Computer Science, Vol. 11714, Springer, 297–316. DOI : https://doi.org/10.1007/978-3-030-29026-9_17
- [21] Jérôme Fortier and Luigi Santocanale. 2013. Cuts for circular proofs: Semantics and cut-elimination. In *Proceedings of the Computer Science Logic 2013 (CSL '13)*, Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik.
 - [22] Martin Hofmann. 1997. A mixed modal/linear lambda calculus with applications to Bellantoni-Cook safe recursion. In *Proceedings of the 11th International Workshop Computer Science Logic (CSL '97), Annual Conference of the EACSL*, Mogens Nielsen and Wolfgang Thomas (Eds.), Lecture Notes in Computer Science, Vol. 1414, Springer, 275–294. DOI : <https://doi.org/10.1007/BFb0028020>
 - [23] Stephen Cole Kleene. 1971. *Introduction to Metamathematics (7 ed.)*. Wolters-Noordhoff Publishing.
 - [24] Leszek Aleksander Kolodziejczyk, Henryk Michalewski, Pierre Pradic, and Michal Skrzypczak. 2019. The logical strength of Büchi's decidability theorem. *Logical Methods in Computer Science* 15, 2 (2019), 16:1–16:31. [https://doi.org/10.23638/LMCS-15\(2:16\)2019](https://doi.org/10.23638/LMCS-15(2:16)2019)
 - [25] Denis Kuperberg, Laureline Pinault, and Damien Pous. 2021. Cyclic proofs, system t, and the power of contraction. *Proceedings of the ACM Programming Language* 5, POPL (2021), 1–28. DOI : <https://doi.org/10.1145/3434282>
 - [26] Daniel Leivant. 1991. A foundational delineation of computational feasibility. In *Proceedings of the 6th Annual Symposium on Logic in Computer Science (LICS '91)*. IEEE Computer Society, 2–11. DOI : <https://doi.org/10.1109/LICS.1991.151625>
 - [27] Daniel Leivant. 1999. Ramified recurrence and computational complexity III: Higher type recurrence and elementary complexity. *Annals of Pure and Applied Logic* 96, 1–3 (1999), 209–229. DOI : [https://doi.org/10.1016/S0168-0072\(98\)00040-2](https://doi.org/10.1016/S0168-0072(98)00040-2)
 - [28] Daniel Leivant and Jean-Yves Marion. 1994. Ramified recurrence and computational complexity II: Substitution and poly-space. In *Proceedings of the 8th International Workshop Computer Science Logic (CSL '94)*, Leszek Pacholski and Jerzy Tiuryn (Eds.), Lecture Notes in Computer Science, Vol. 933, Springer, 486–500. DOI : <https://doi.org/10.1007/BFb0022277>
 - [29] Grigori E. Mints. 1978. Finite investigations of transfinite derivations. *Journal of Soviet Mathematics* 10, 4 (1978), 548–596.
 - [30] Damian Niwiński and Igor Walukiewicz. 1996. Games for the μ -calculus. *Theoretical Computer Science* 163, 1–2 (1996), 99–116.
 - [31] Rémi Nollé, Alexis Saurin, and Christine Tasson. 2019. PSPACE-completeness of a thread criterion for circular proofs in linear logic with least and greatest fixed points. In *Proceedings of the 28th International Conference on Automated Reasoning with Analytic Tableaux and Related Methods (TABLEAUX '19)*, Serenella Cerrito and Andrei Popescu (Eds.), Lecture Notes in Computer Science, Vol. 11714, Springer, 317–334. DOI : https://doi.org/10.1007/978-3-030-29026-9_18
 - [32] Isabel Oitavem. 2008. Characterizing PSPACE with pointers. *Mathematical Logic Quarterly* 54, 3 (2008), 323–329. DOI : <https://doi.org/10.1002/malq.200610056>
 - [33] H. E. Rose. 1984. *Subrecursion: Functions and Hierarchies*. Oxford University Press.
 - [34] Alex Simpson. 2017. Cyclic arithmetic is equivalent to peano arithmetic. In *Proceedings of the 20th International Conference on Foundations of Software Science and Computation Structures (FOSSACS '17), Held as Part of the European Joint Conferences on Theory and Practice of Software (ETAPS '17)*, Javier Esparza and Andrzej S. Murawski (Eds.), Lecture Notes in Computer Science, Vol. 10203, 283–300. DOI : https://doi.org/10.1007/978-3-662-54458-7_17
 - [35] Marc Wirz. 1999. *Characterizing the Grzegorzczak Hierarchy by Safe Recursion*. Universität Bern.

Appendices

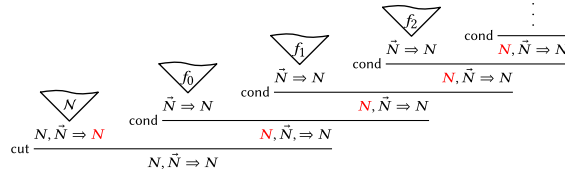
A Computational Expressivity of (Progressing) Coderivations

A.1 Examples of Coderivations for Proposition 20

In light of Proposition 19, we shall simply omit modalities in (regular) (progressing) coderivations in what follows, i.e., we shall regard any formula in the antecedent of a sequent as modal and we shall omit applications of $\Box_r$. Consequently, we shall write, e.g., cut instead of $\text{cut}_\Box$ (and so on) and avoid writing semicolons in the semantics of a coderivation.

From here, *Items 2 and 1* from Proposition 20 are proved by way of the following examples.

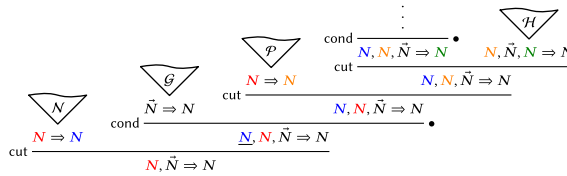
Example A1 (Extensional Completeness at Type 1). For any function $f : \mathbb{N}^k \rightarrow \mathbb{N}$ there is a progressing coderivation $\mathcal{F}$ such that $f_{\mathcal{F}} = f$. Proceeding by induction on k , if $k = 0$ then we use the rules $0, s_0, s_1$, cut to construct $\mathcal{F}$ defining the natural number f . Otherwise, suppose $f : \mathbb{N} \times \mathbb{N}^k \rightarrow \mathbb{N}$ and define f_n as $f_n(\vec{x}) = f(n, \vec{x})$. We construct the coderivation defining f as follows:



where $\mathcal{N}$ is a coderivation converting n into $s_1.^n.s_1 0$, and we omit the second premise of cond as it is never selected. Notice that $\mathcal{F}$ is progressing, as red formulas N (which are modal) form a progressing thread.

The above example illustrates the role of regularity as a *uniformity* condition: regular coderivations admit a finite description (e.g., a finite tree with backpointers), so that they define computable functions. In fact, it turns out that any (partial) recursive function is B^- -definable by a regular coderivation. This can be easily inferred from the next two examples using Proposition 19.

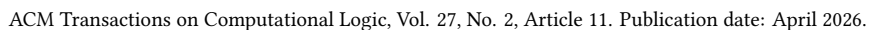
Example A2 (Primitive Recursion). Let $f(x, \vec{x})$ be defined by primitive recursion from $g(\vec{x})$ and $h(x, \vec{x}, y)$. Given coderivations $\mathcal{G}$ and $\mathcal{H}$ defining g and h respectively, we construct the following coderivation $\mathcal{R}$:



where $\mathcal{N}$ is the coderivation converting n into $s_1.^n.s_1 0$, $\mathcal{P}$ is a coderivation defining unary predecessor, and we avoid writing the second premise of cond as it is never selected. Notice that $\mathcal{R}$ is progressing, as blue formulas N (which are modal) form a progressing thread contained in the infinite branch that loops on $\bullet$. From the associated equational program, we obtain:

$$\begin{aligned} f_{\mathcal{R}_e}(x, \vec{x}) &= f_{\mathcal{R}_1}(f_{\mathcal{N}}(x), x, \vec{x}) \\ f_{\mathcal{R}_1}(0, x, \vec{x}) &= g(\vec{x}) \\ f_{\mathcal{R}_1}(s_1 z, x, \vec{x}) &= h(f_{\mathcal{P}}(x), \vec{x}, f_{\mathcal{R}_1}(z, f_{\mathcal{P}}(x), \vec{x})), \end{aligned}$$

Example A3 (Unbounded Search). Let $g(x, \vec{x})$ be a function, and let $f(\vec{x}) := \mu x.(g(x, \vec{x}) = 0)$ be the unbounded search function obtained by applying the minimisation operation on $g(\vec{x})$. Given a coderivation $\mathcal{G}$ defining g , we construct the following coderivation $\mathcal{U}$:



Given a function $f(\vec{x}) \in T_1(\vec{a})$, we construct a regular progressing coderivation of B^- ,

$$\frac{\left\{ a_i \frac{}{\vec{N} \Rightarrow N} \right\}_i}{\mathcal{D}_f(\vec{a})} \vec{N} \Rightarrow N$$

computing $f(\vec{x})$ over $\vec{a}$, by induction on the definition of $f(\vec{x})$.

If $f(\vec{x})$ is an initial function, an oracle, or $f(\vec{x}) = h(g(\vec{x}), \vec{x})$ then the construction is easy (see, e.g., the Proof of Theorem 55).

Let us consider the case of recursion (which subsumes usual primitive recursion at type 0). Suppose $f(x, \vec{x}) \in T_1(\vec{a})$ where,

$$\begin{aligned} f(0, \vec{x}) &= g(\vec{x}) \\ f(s_0 x, \vec{x}) &= h_0(\lambda \vec{u}. f(x, \vec{u}))(x, \vec{x}) \\ f(s_1 x, \vec{x}) &= h_1(\lambda \vec{u}. f(x, \vec{u}))(x, \vec{x}), \end{aligned}$$

where $\mathcal{D}_g(\vec{a})$ and $\mathcal{D}_h(a, \vec{a})$ are already obtained by the inductive hypothesis. We define $\mathcal{D}_f(\vec{a})$ as follows:

$$\frac{\begin{array}{c} \vdots \\ \frac{}{N, \vec{N} \Rightarrow N} \bullet \\ \mathcal{D}_g(\vec{a}) \\ \vec{N} \Rightarrow N \end{array} \quad \frac{\begin{array}{c} \vdots \\ \frac{}{N, \vec{N} \Rightarrow N} \bullet \\ N, \mathcal{D}_{h_0}(\mathcal{D}_f(\vec{a}), \vec{a}) \\ N, \vec{N} \Rightarrow N \end{array} \quad \frac{\begin{array}{c} \vdots \\ \frac{}{N, \vec{N} \Rightarrow N} \bullet \\ N, \mathcal{D}_{h_1}(\mathcal{D}_f(\vec{a}), \vec{a}) \\ N, \vec{N} \Rightarrow N \end{array}}{\text{cond} \frac{}{N, \vec{N} \Rightarrow N} \bullet}$$

Note that the existence of the second and third coderivations above the conditional is given by a similar construction to that of Lemma 54 (see also [15, Lemma 42]).

A.3 On the ‘Power’ of Contraction

Given the computationally equivalent system CT_0 with contraction from [15], we can view the above result as a sort of ‘contraction admissibility’ for regular progressing B^- -coderivations. Let us take a moment to make this formal.

Call $B^- + \{c_N, c_{\square N}\}$ the extension of B^- with the rules c_N and $c_{\square N}$ below:

$$c_N \frac{\Gamma, N, N \Rightarrow B}{\Gamma, N \Rightarrow B} \quad c_{\square N} \frac{\Gamma, \square N, \square N \Rightarrow B}{\Gamma, \square N \Rightarrow B}, \quad (23)$$

where the semantics for the new system extends the one for B^- in the obvious way, and the notion of (progressing) thread is induced by the given colouring.⁸ We have:

COROLLARY 66. *$f(\vec{x}; \cdot)$ is definable by a regular progressing $B^- + \{c_N, c_{\square N}\}$ -coderivation iff it is definable by a regular progressing B^- -coderivation.*

⁸Note that the totality argument of Proposition 16 still applies in the presence of these rules, cf. also [15].

Remark 67. The reader may at this point wonder if a direct ‘contraction-admissibility’ argument exists for the rules in (23). First, notice that c_N can be derived in B^- :

While a similar derivation exists for $c_{\square}$, note that this crucially does not preserve the same notion of thread (cf. colours above) and so does not, *a priori*, preserve progressiveness.

Fig. A1. Ackermann-Péter function computed by a regular progressing B^- -coderivation (without explicit contractions).

$$\begin{aligned} A(0, y) &= x + 1 \\ A(x + 1, 0) &= A(x, 1) \\ A(x + 1, y + 1) &= A(x, A(x + 1, y)). \end{aligned}$$

The usual cyclic representation of $A(x, y)$ using only base types, e.g., as in [15, 25], mimics the equational program above, namely applying case analysis on the first input before applying case analysis on the second input. It is important in this construction to be able to explicitly contract the first input, corresponding to x , due to the third line of the equational program above.

The verification of the semantics of this coderivation is routine. To see that it is progressing, we may conduct a case analysis on the set L of infinitely often visited loops, among (1), (2), (3), in an arbitrary branch:

- ACM Transactions on Computational Logic, Vol. 27, No. 2, Article 11. Publication date: April 2026.

- if $L = \{(3)\}$ then there is a progressing thread on the red N ;
- if $L = \{(1), (2)\}$ then there is a progressing thread on the red N : on each iteration of (1) the thread progresses, and remains intact (but does not progress) on each iteration of (2);
- if $L = \{(1), (3)\}$ then there is a progressing thread on the red N : the thread progresses on each iteration of either (1) or (3);
- if $L = \{(2), (3)\}$ then there is a progressing thread on the red N : on each iteration of (2) the thread remains intact (but does not progress), and on each iteration of (3) the thread progresses;
- if $L = \{(1), (2), (3)\}$ then there is a progressing thread on the red N : the thread progresses on each iteration of (1) or (3), and remains intact (but does not progress) on each iteration of (3).

Received 30 April 2024; revised 3 September 2025; accepted 10 January 2026